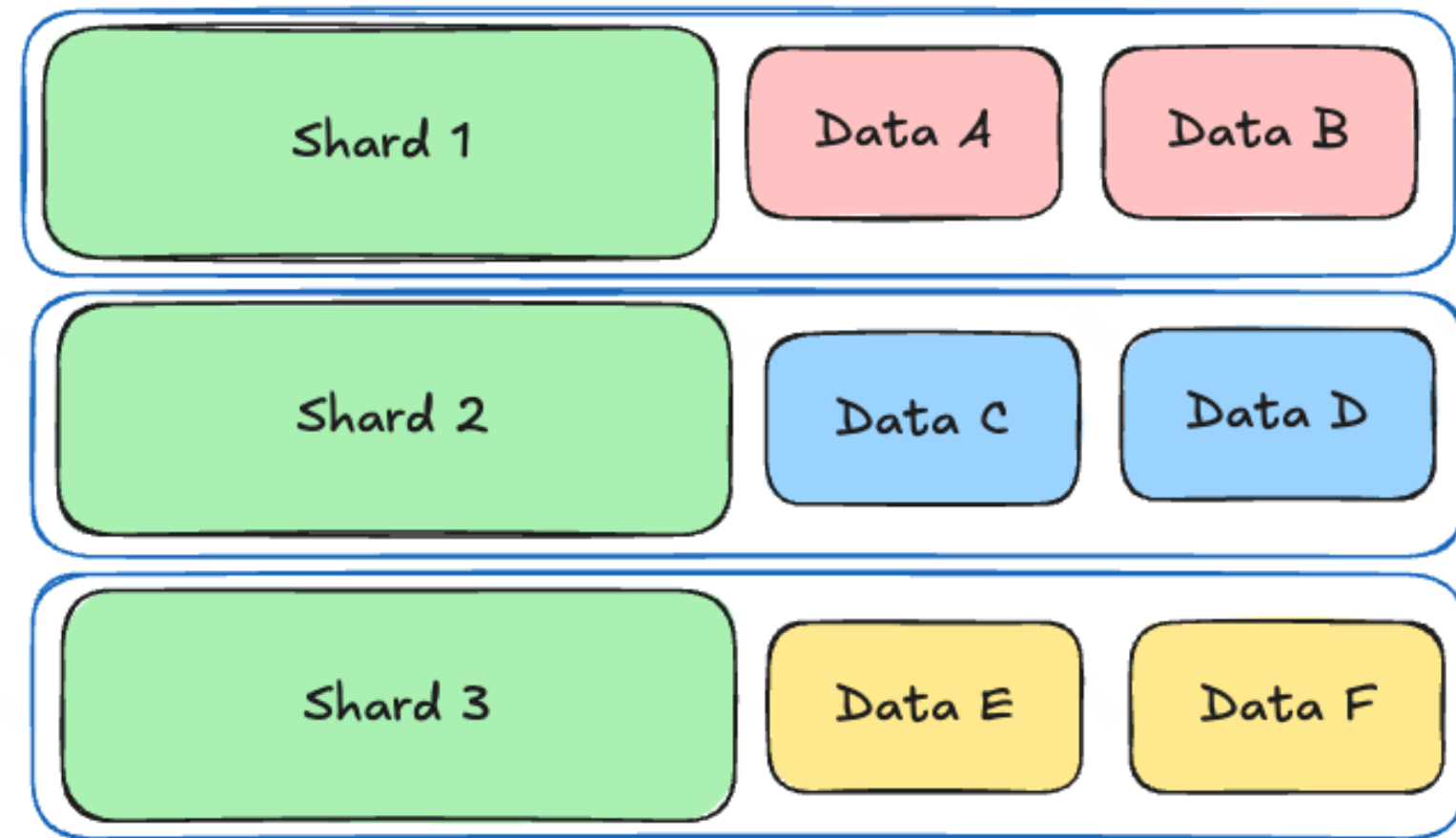


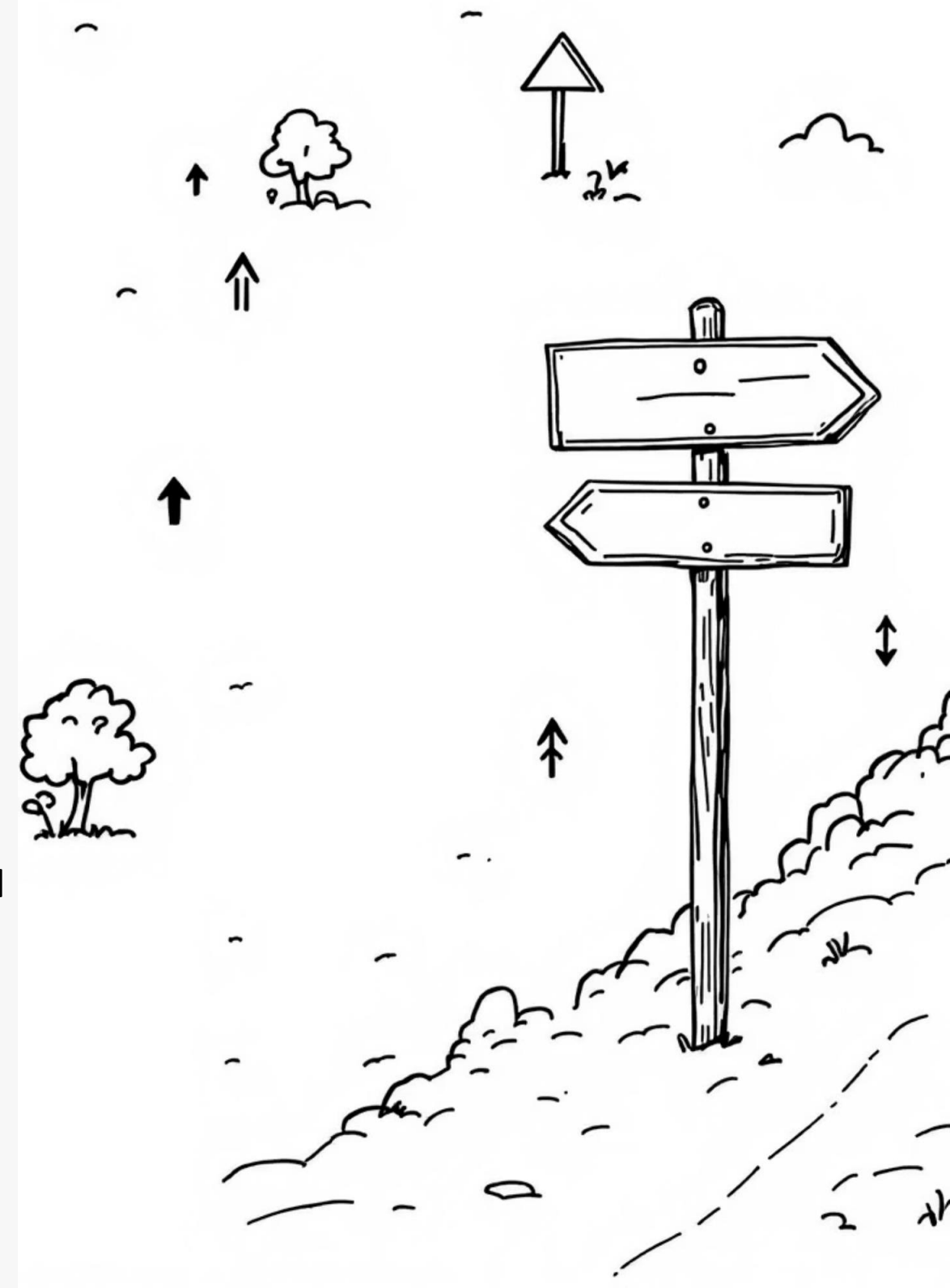
Проектирование Распределенных Систем

Восьмой семинар:
Шардирование и консистентное хеширование



План семинара

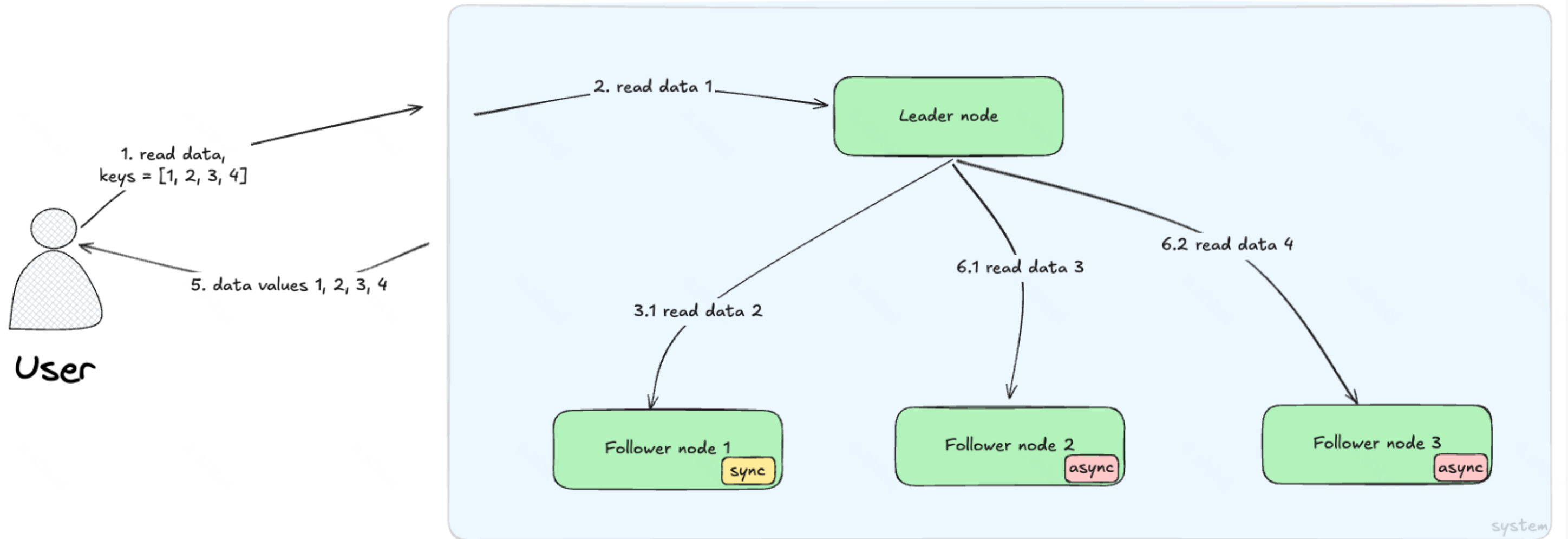
- Основы шардирования
- Требования к функции шардирования
- Подходы к шардированию.
Стратегии разбиения данных
- Стратегии выбора количества шардов
- Согласованное хеширование
- Later:
 - Альтернативные подходы к хешированию
 - Проблемы горячих шардов. Балансировка нагрузки
 - Маршрутизация запросов
 - Комбинирование шардирования и репликации



О чем говорили в прошлый раз?

О чем говорили в прошлый раз?

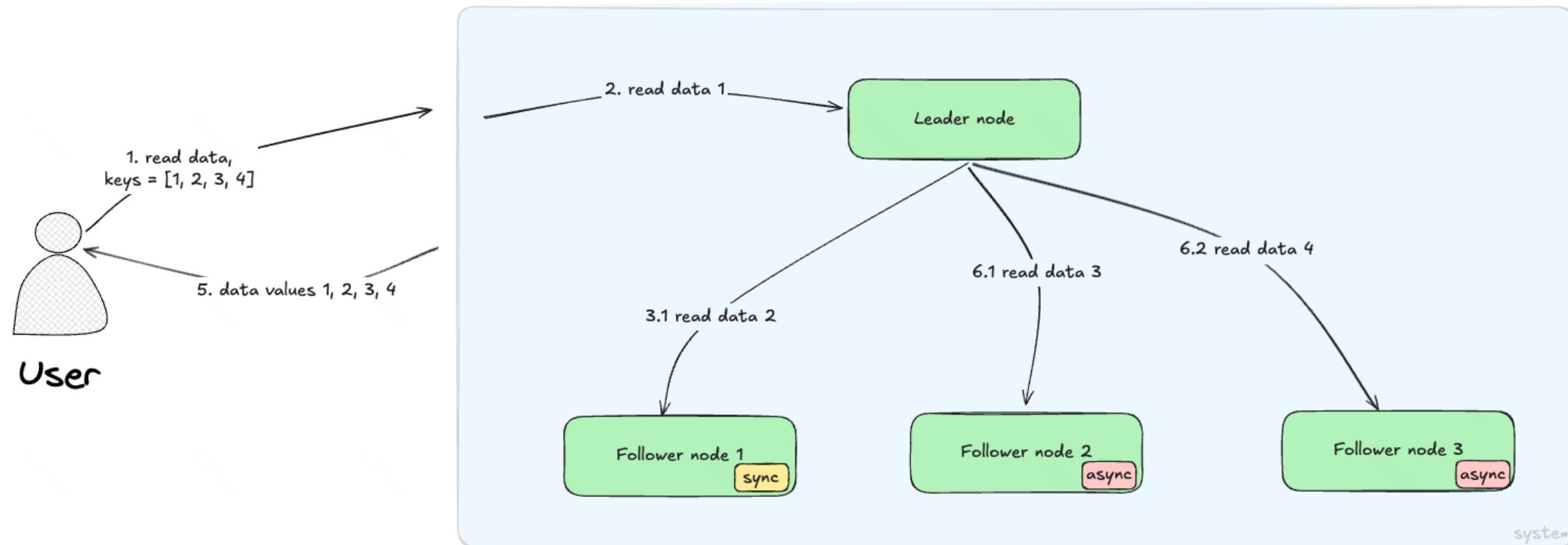
Репликация



О чем говорили в прошлый раз?

Репликация копирует данные на несколько узлов.

Все реплики содержат одни и те же данные



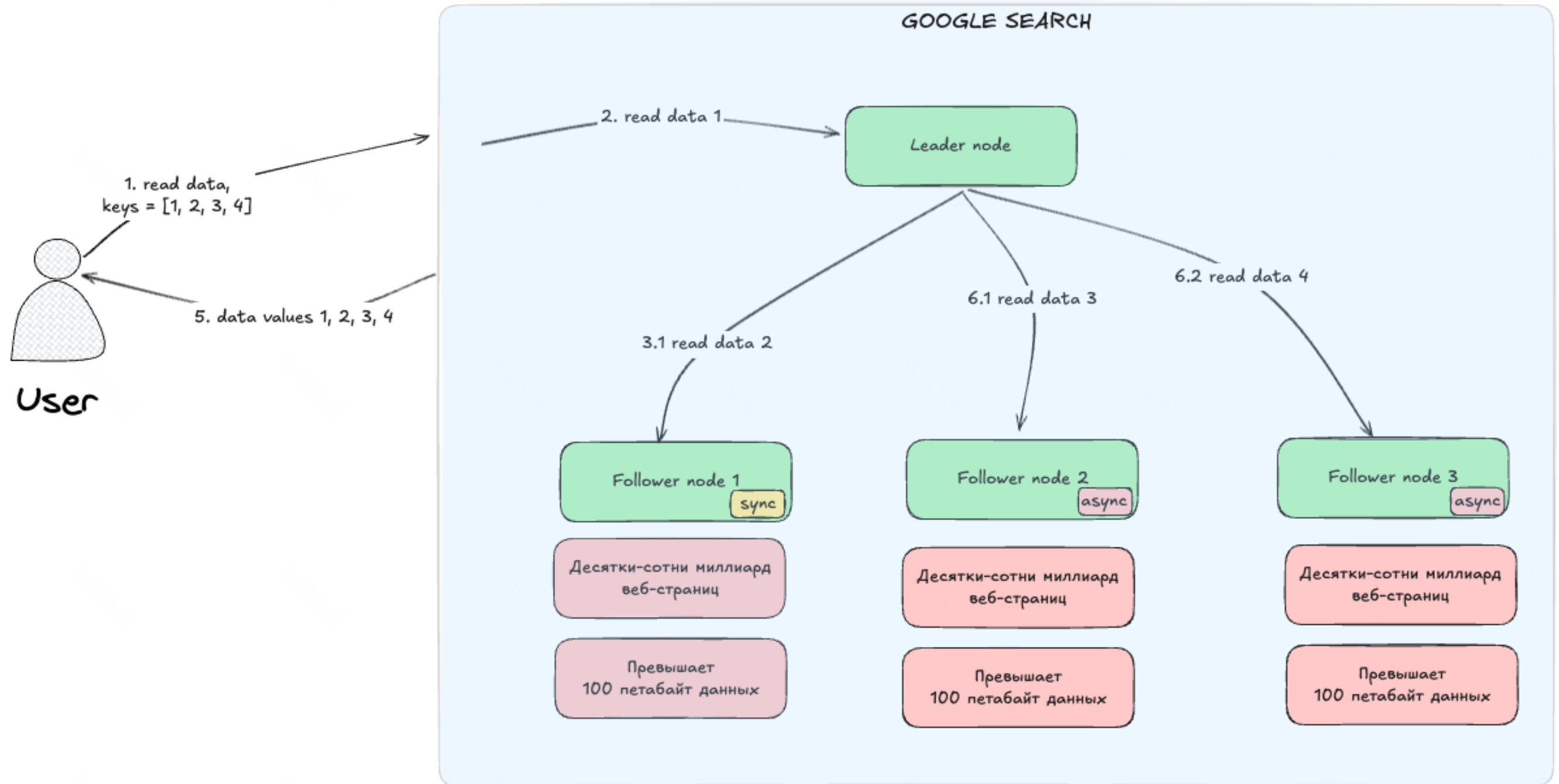
Возникают проблемы

Главная проблема при
использовании **ТОЛЬКО**
репликации

Возникают проблемы



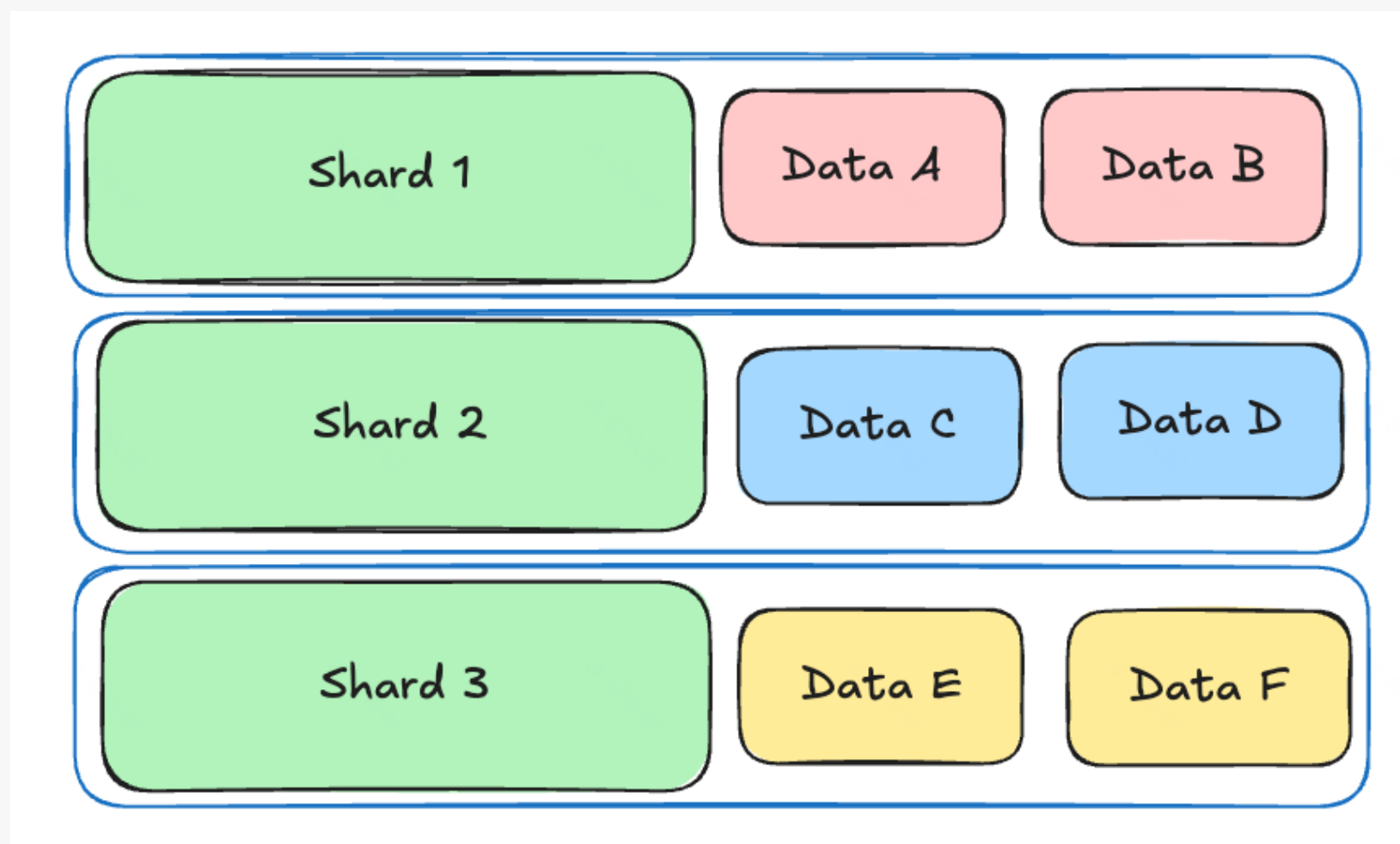
Возникают проблемы (пример Google Search)



А что если хранить на одной ноде не все данные, а только необходимую часть?

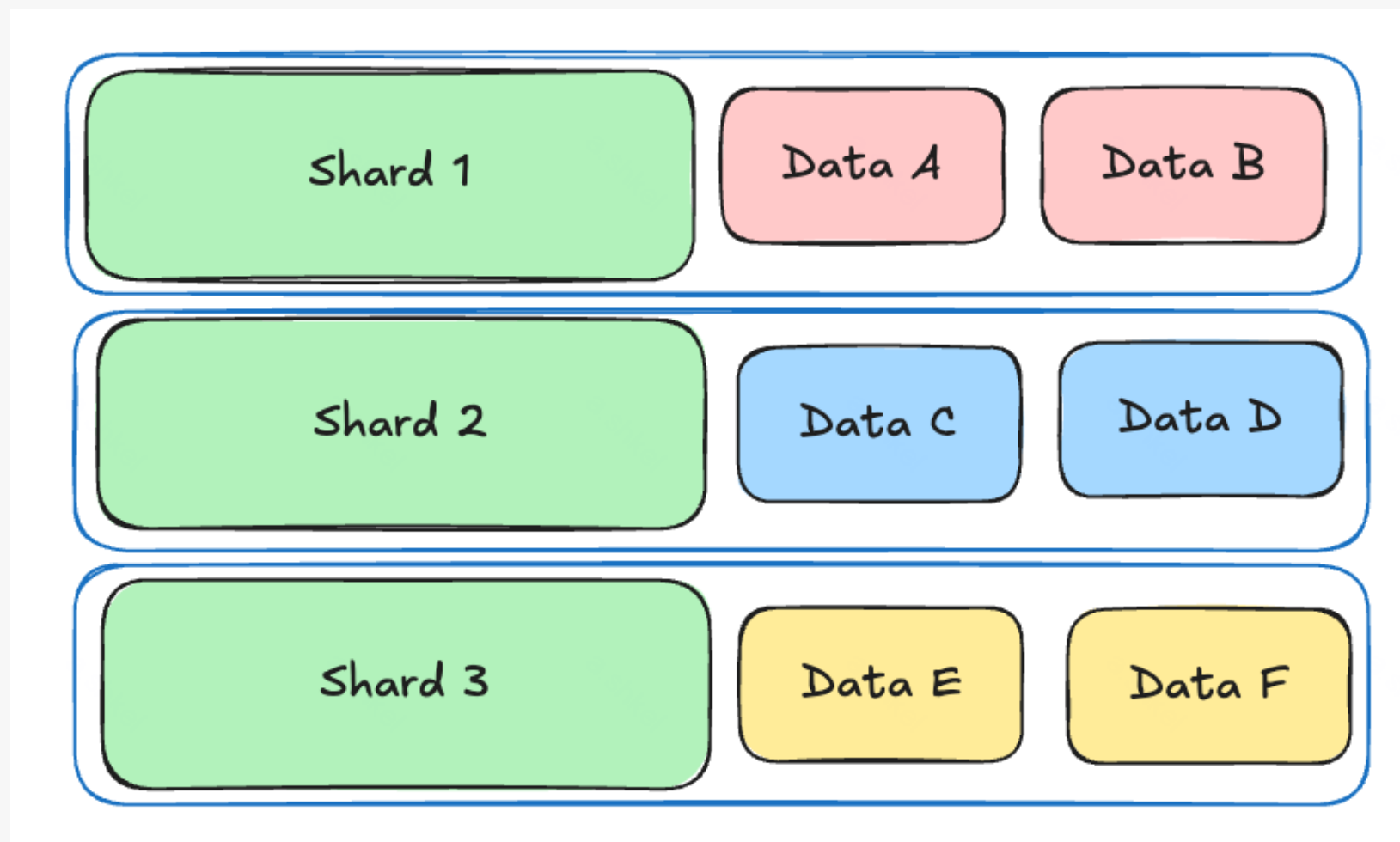
Шардирование

Шардирование (Partitioning) – это техника разделения большого набора данных на несколько независимых подмножеств (шардов), где каждый шард может быть обработан отдельным узлом.



Шардирование (Partitioning) – это техника разделения большого набора данных на несколько независимых подмножеств (шардов), где каждый шард может быть обработан отдельным узлом.

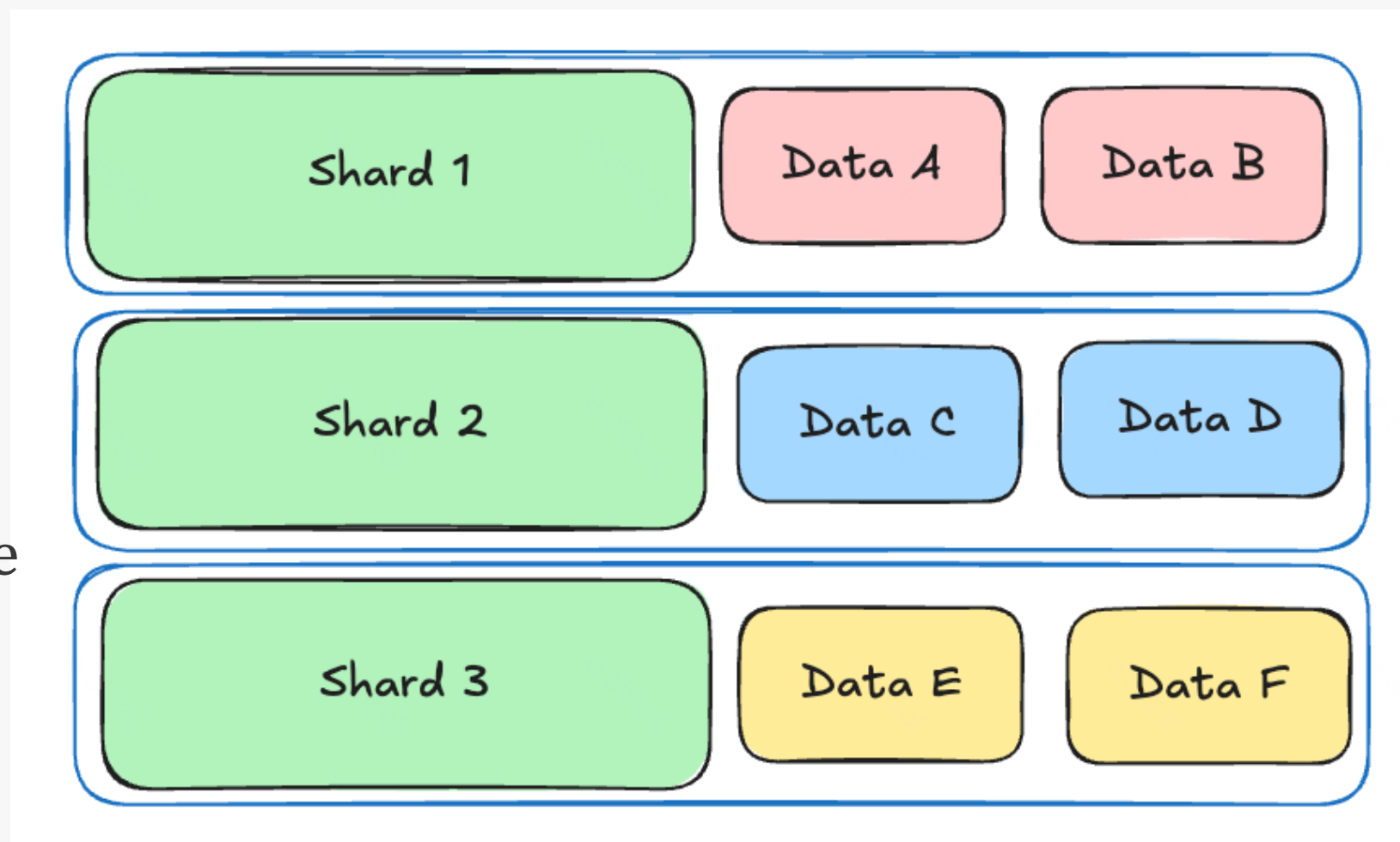
Шард – это независимое подмножество данных, чаще всего находящееся на отдельном узле.



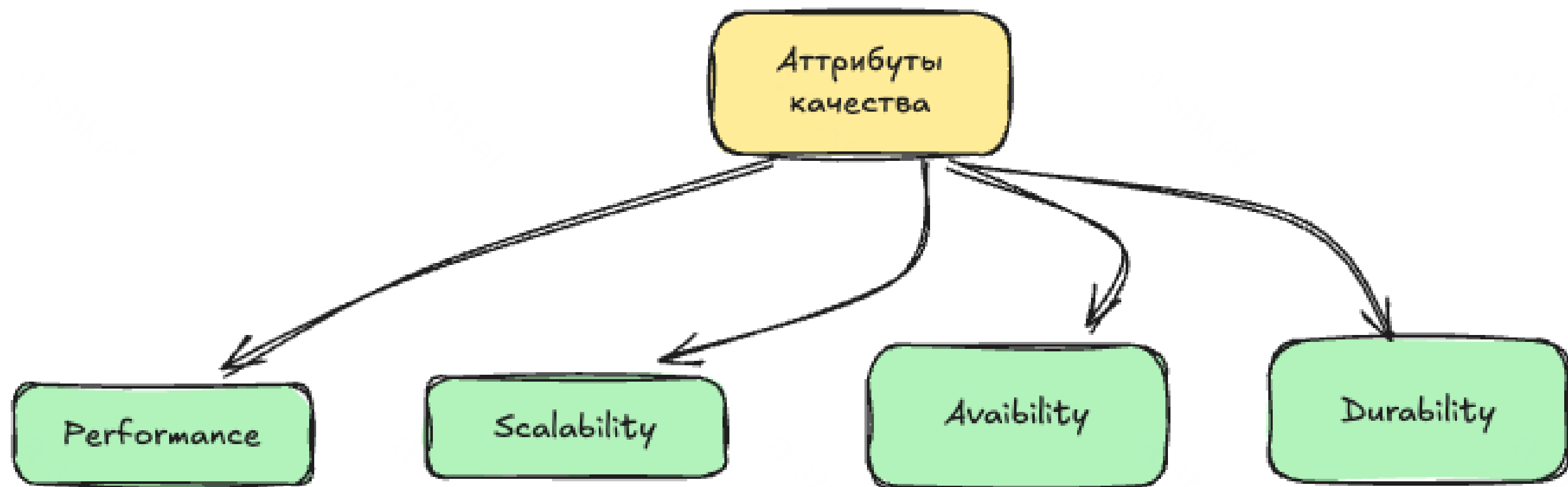
Шардирование (Partitioning) – это техника разделения большого набора данных на несколько независимых подмножеств (шардов), где каждый шард может быть обработан отдельным узлом.

Шард – это независимое подмножество данных, чаще всего находящееся на отдельном узле.

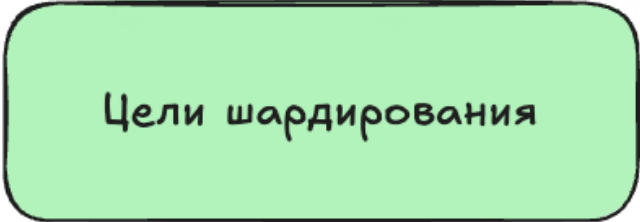
Ключ шардирования (Partition key) – это значение или набор значений, которые определяют, в какой шард должны попасть данные



Вспомним про атрибуты качества системы



Цели шардирования



Цели шардирования

Цели шардирования

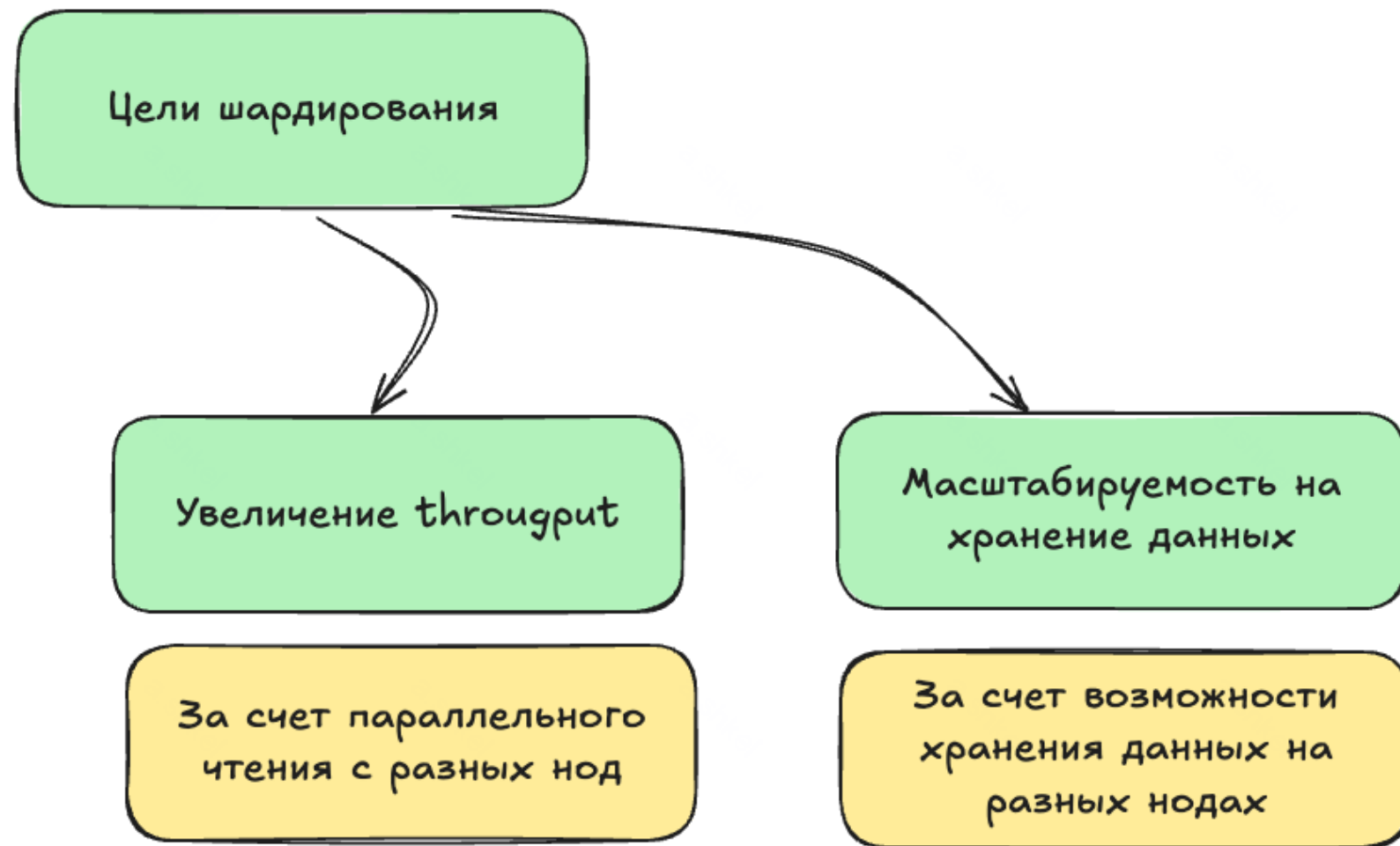
Цели шардирования

```
graph TD; A[Цели шардирования] --> B[Масштабируемость на хранение данных]; B --> C[За счет возможности хранения данных на разных нодах];
```

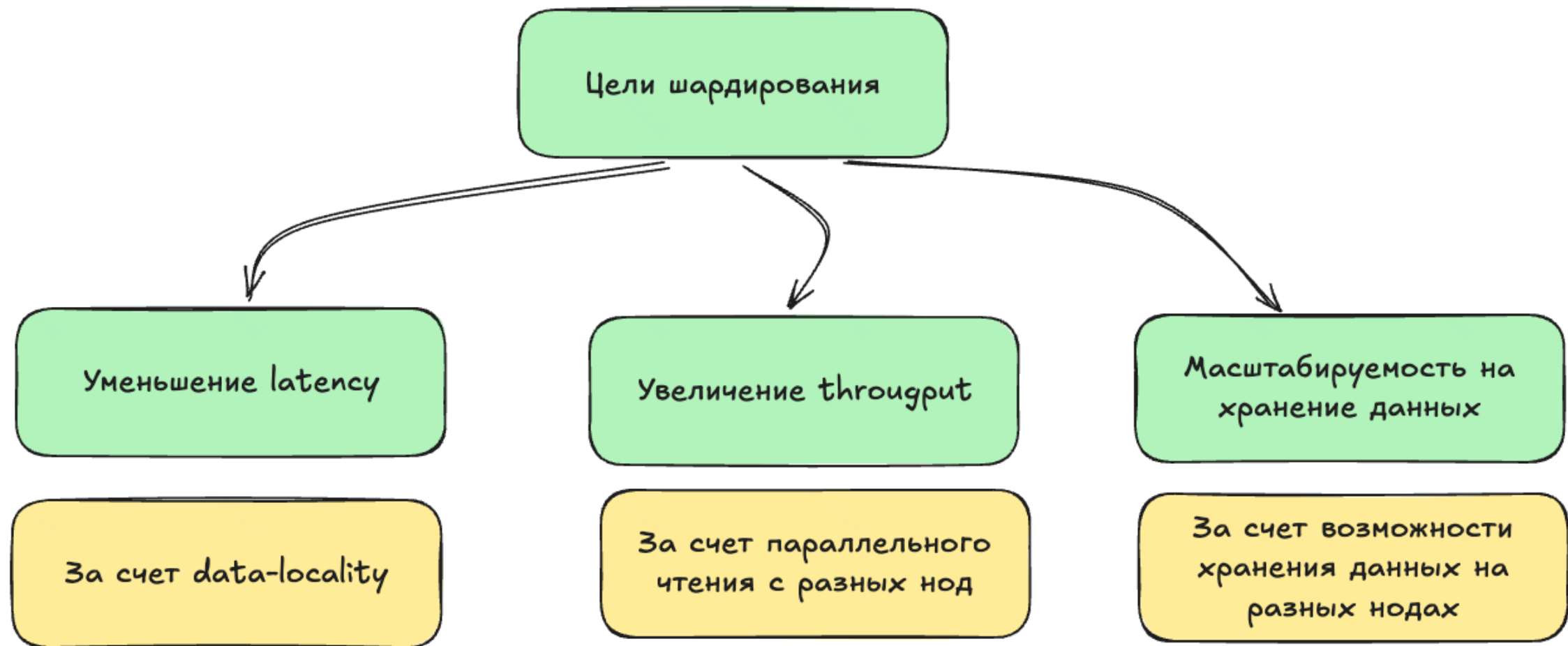
Масштабируемость на
хранение данных

За счет возможности
хранения данных на
разных нодах

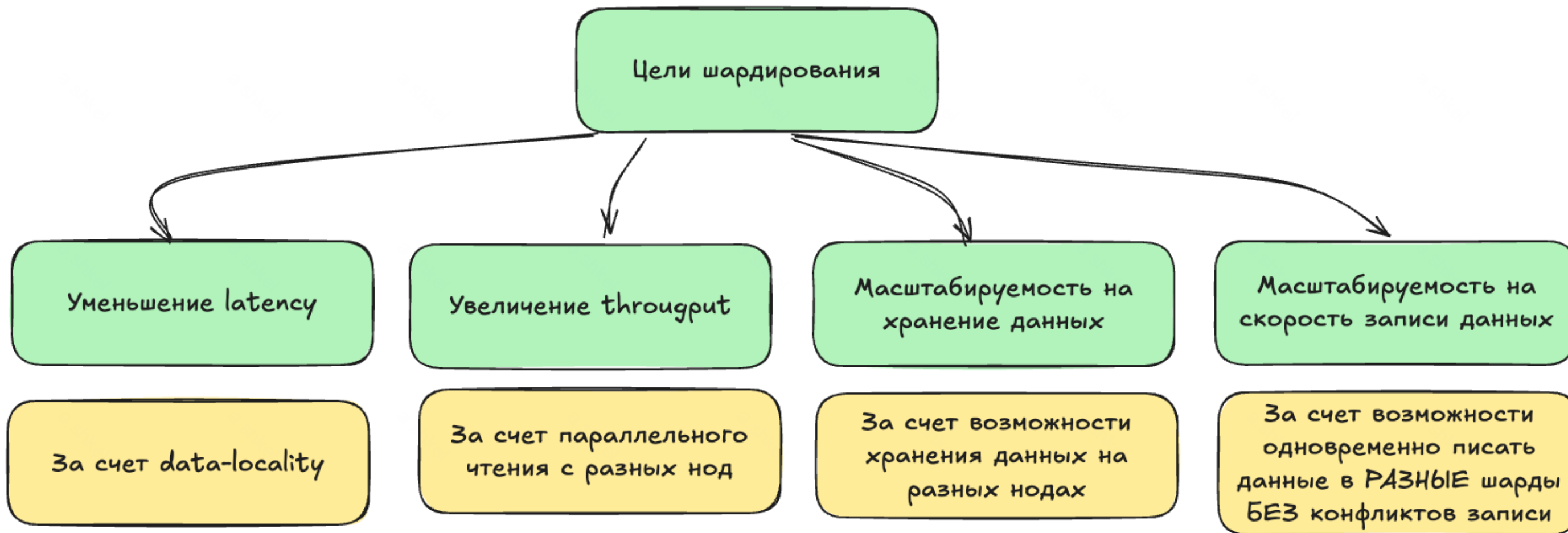
Цели шардирования



Цели шардирования



Цели шардирования

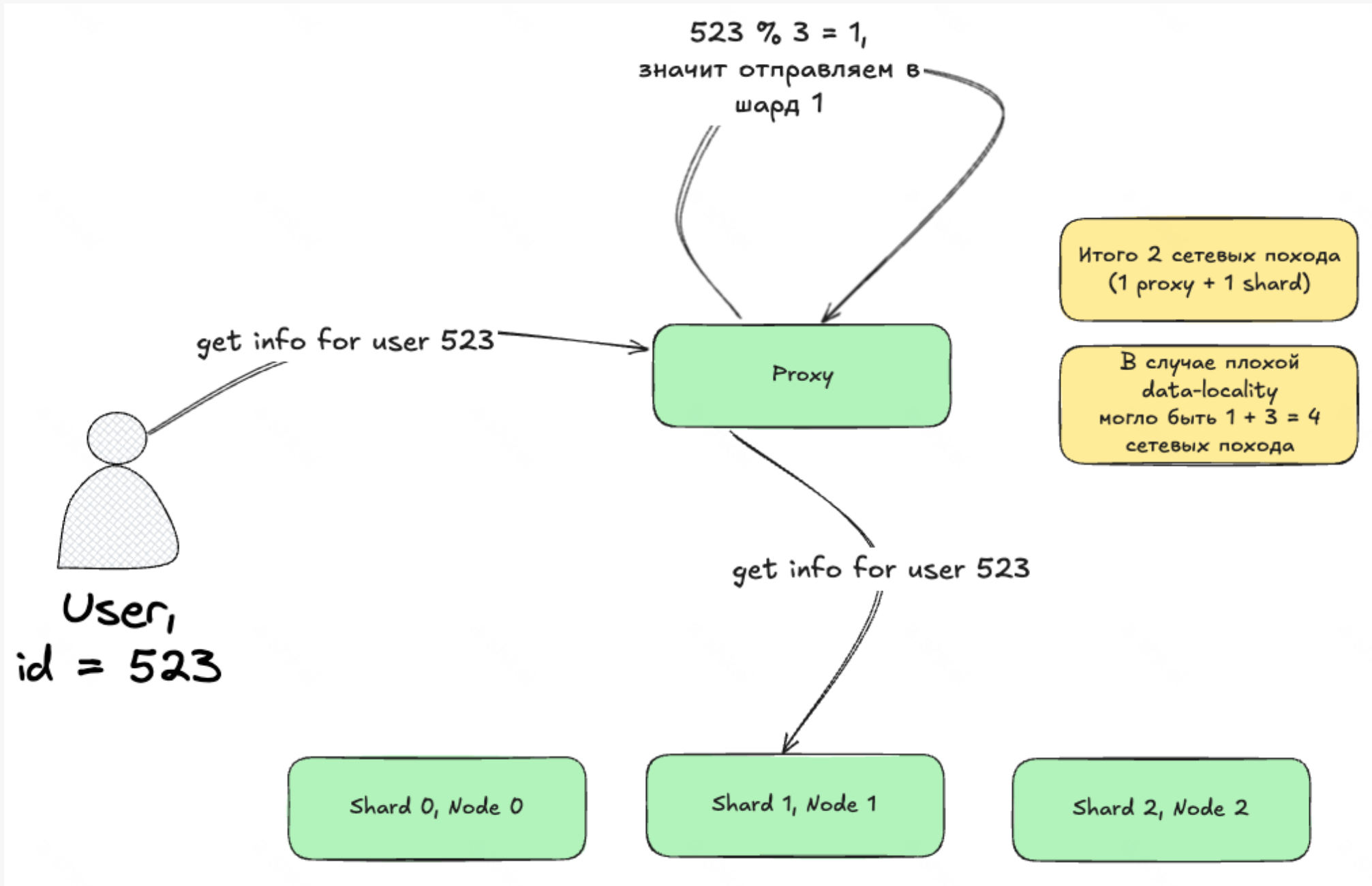


Шардирование vs Репликация

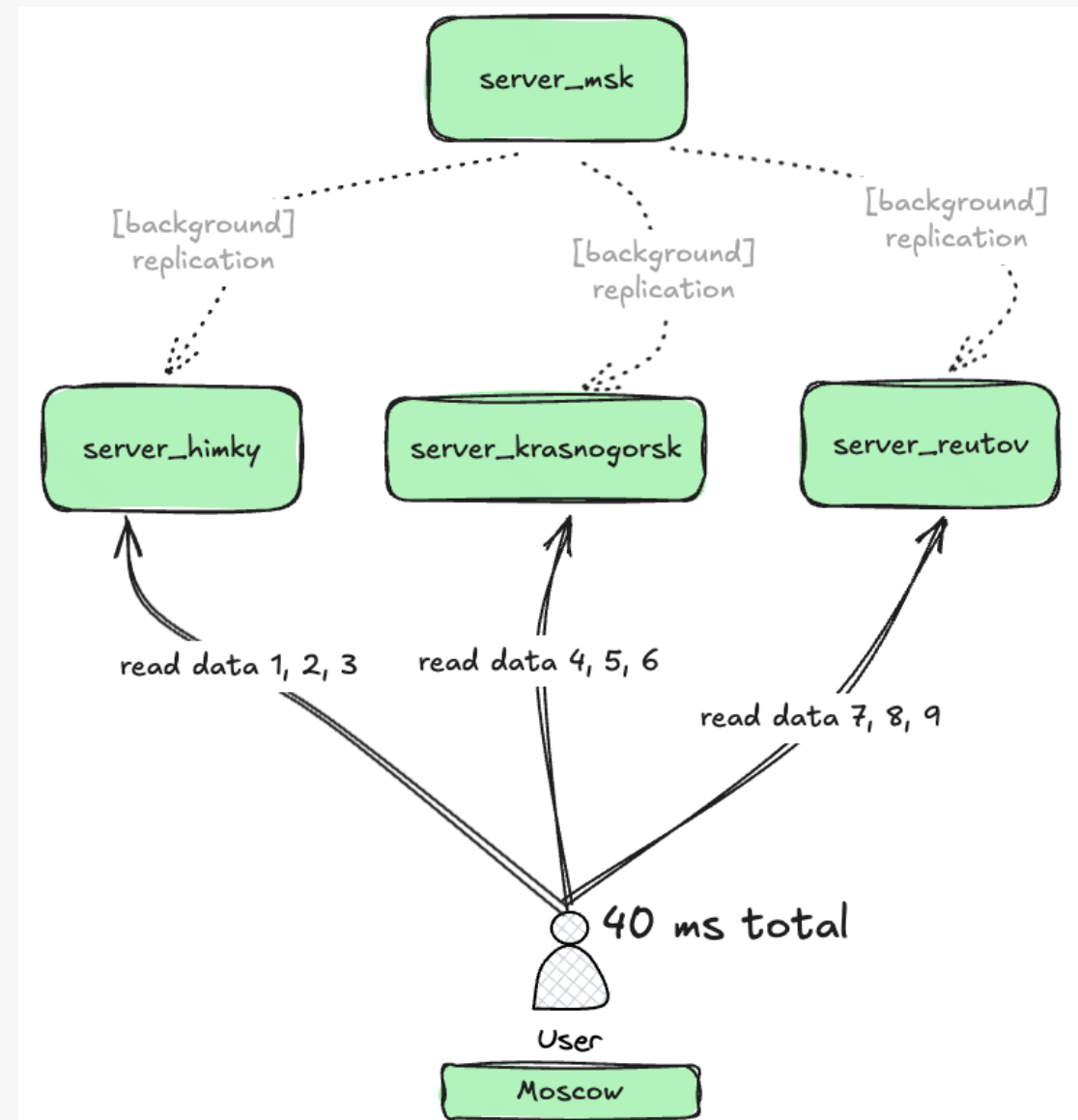
Репликация vs Шардирование:

Аспект	Репликация	Шардирование
Цель	Отказоустойчивость, масштабируемость на чтение	Масштабируемость на хранение данных, масштабируемость на запись и чтение
Данные	Одинаковые копии на разных узлах	Разные части данных на разных узлах
Узлы	Содержат одно и то же	Содержат принципиально разные данные
Масштабируемость записи	Нет (все пишут в одни реплики)	Да (могут писать параллельно в разные шарды)
Если один узел падает	Есть копии	Данные потеряны (нужна репликация поверх шардирования)

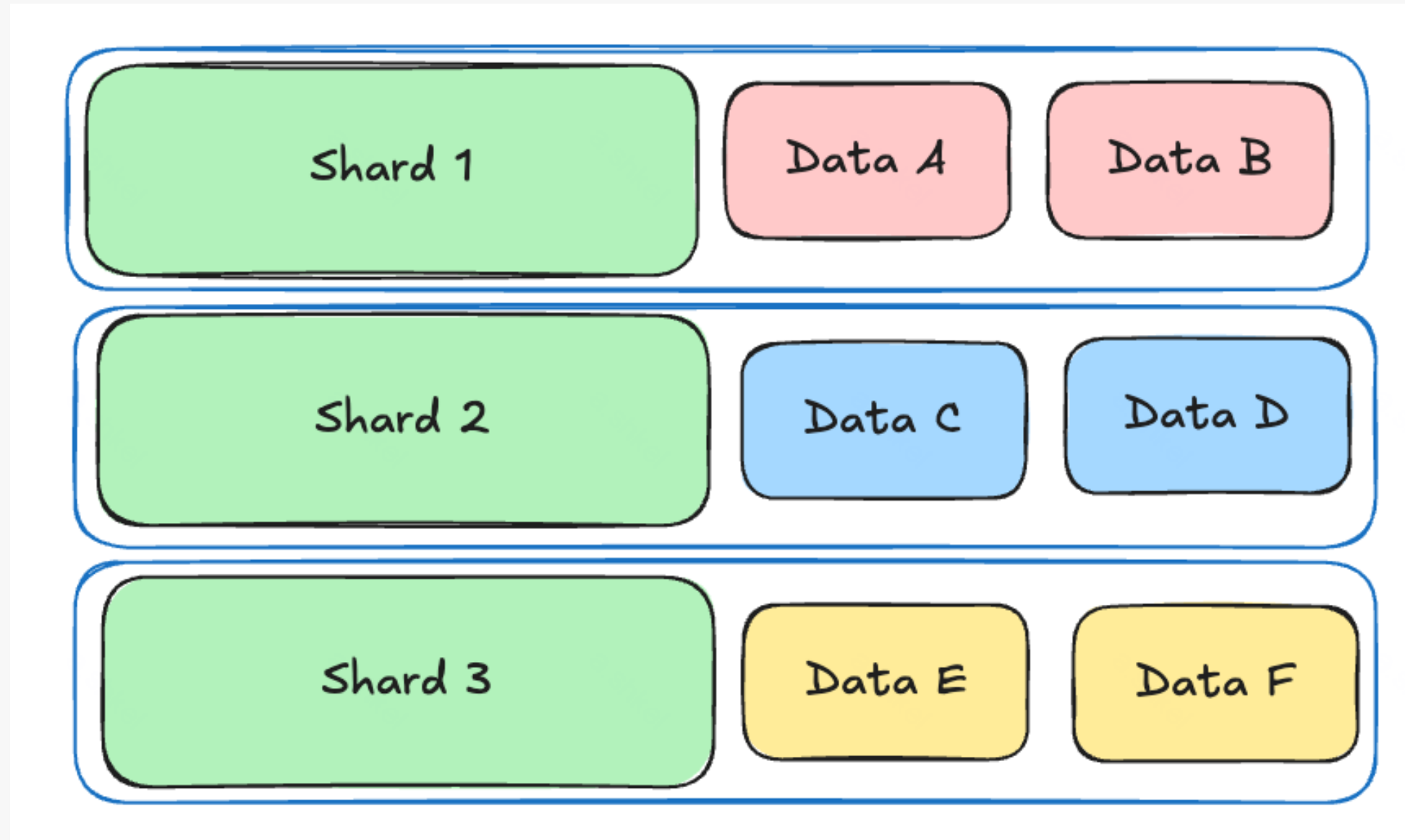
Пример уменьшение latency за счет data-locality



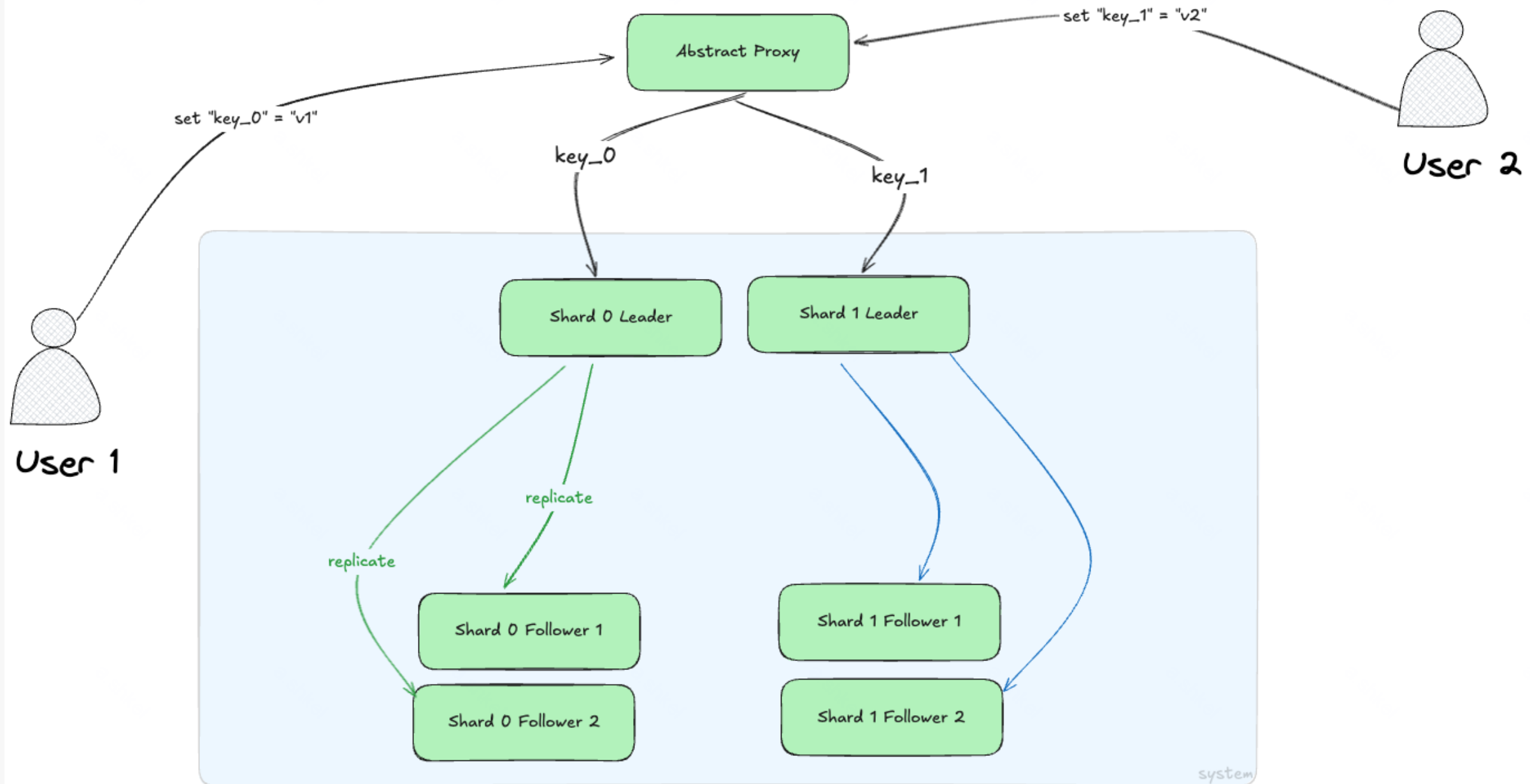
Увеличение throughput системы на чтение: тут
тоже самое, что и с репликацией с:



Пример масштабирования на хранение данных



Пример масштабирования на скорость записи данных



Это все хорошо.

Но как мне разделить данные на шарды?

Функция шардирования (партиционирования)

```
shardId = shardingFunction(key)
```

Функция шардирования (партиционирования)



Требования к функции

Функция шардирования (партиционирования)

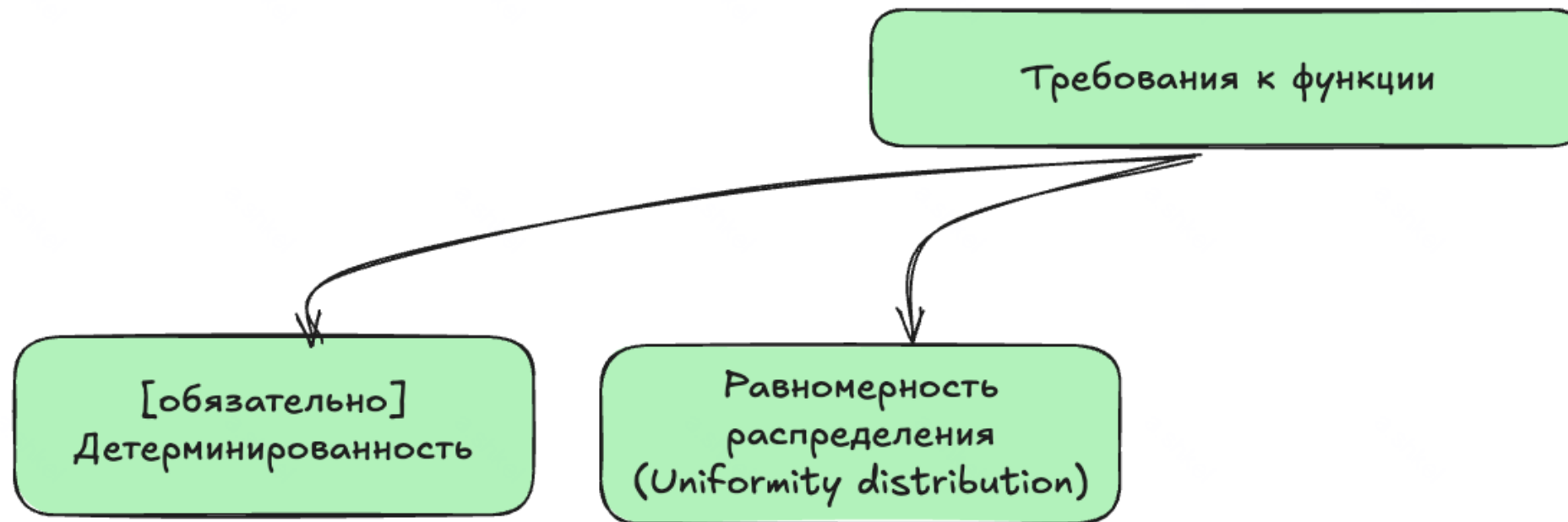
Требования к функции

```
graph TD; A[Требования к функции] --> B["[обязательно]  
Детерминированность"]
```

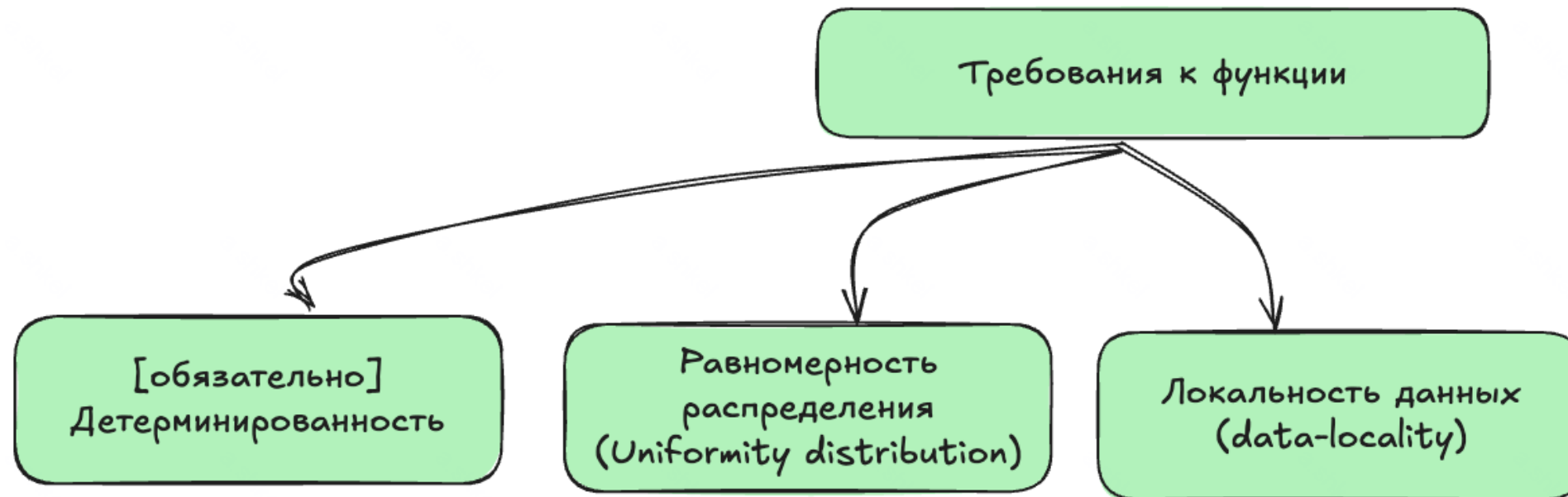
A diagram with two green rounded rectangular boxes. The top box contains the text 'Требования к функции'. A curved arrow points from the bottom of this box to the top of a second box below and to the left. The second box contains the text '[обязательно]' followed by 'Детерминированность' on the next line.

[обязательно]
Детерминированность

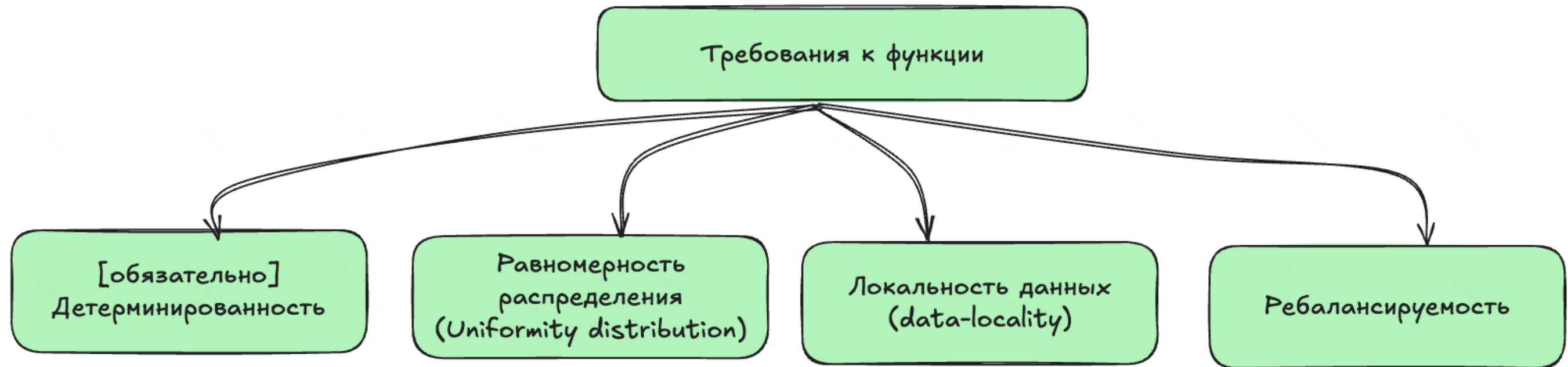
Функция шардирования (партиционирования)



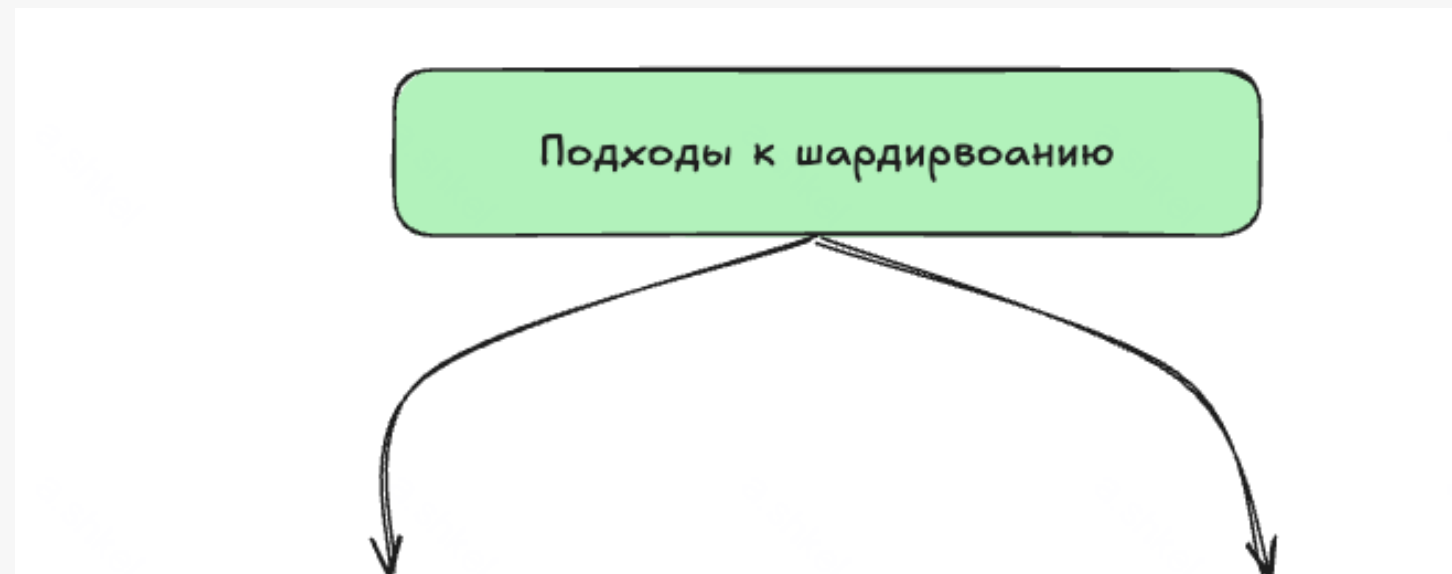
Функция шардирования (партиционирования)



Функция шардирования (партиционирования)



Подходы к шардированию

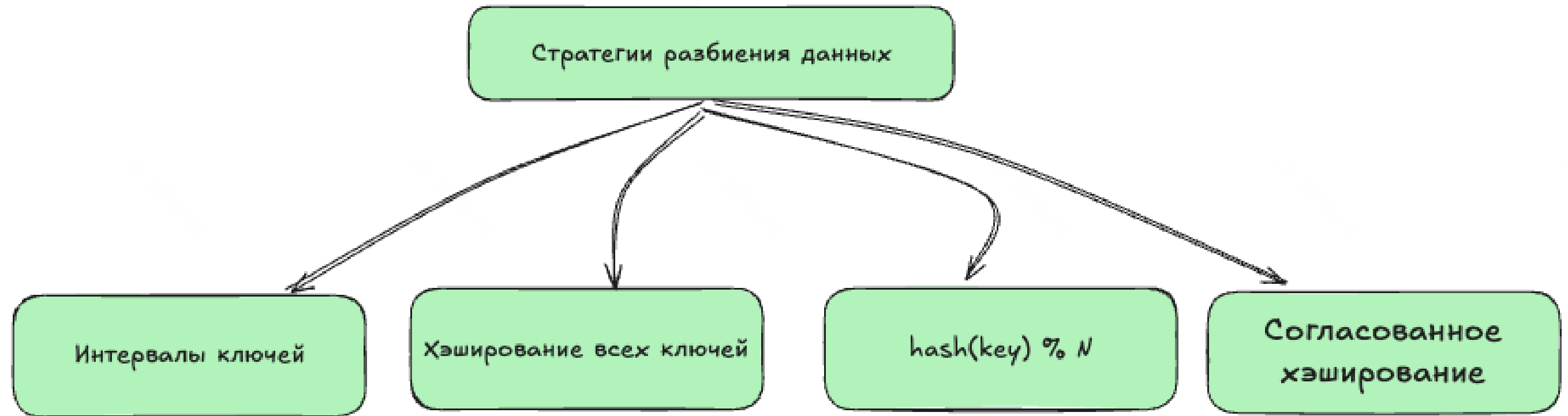


Подходы к шардированию



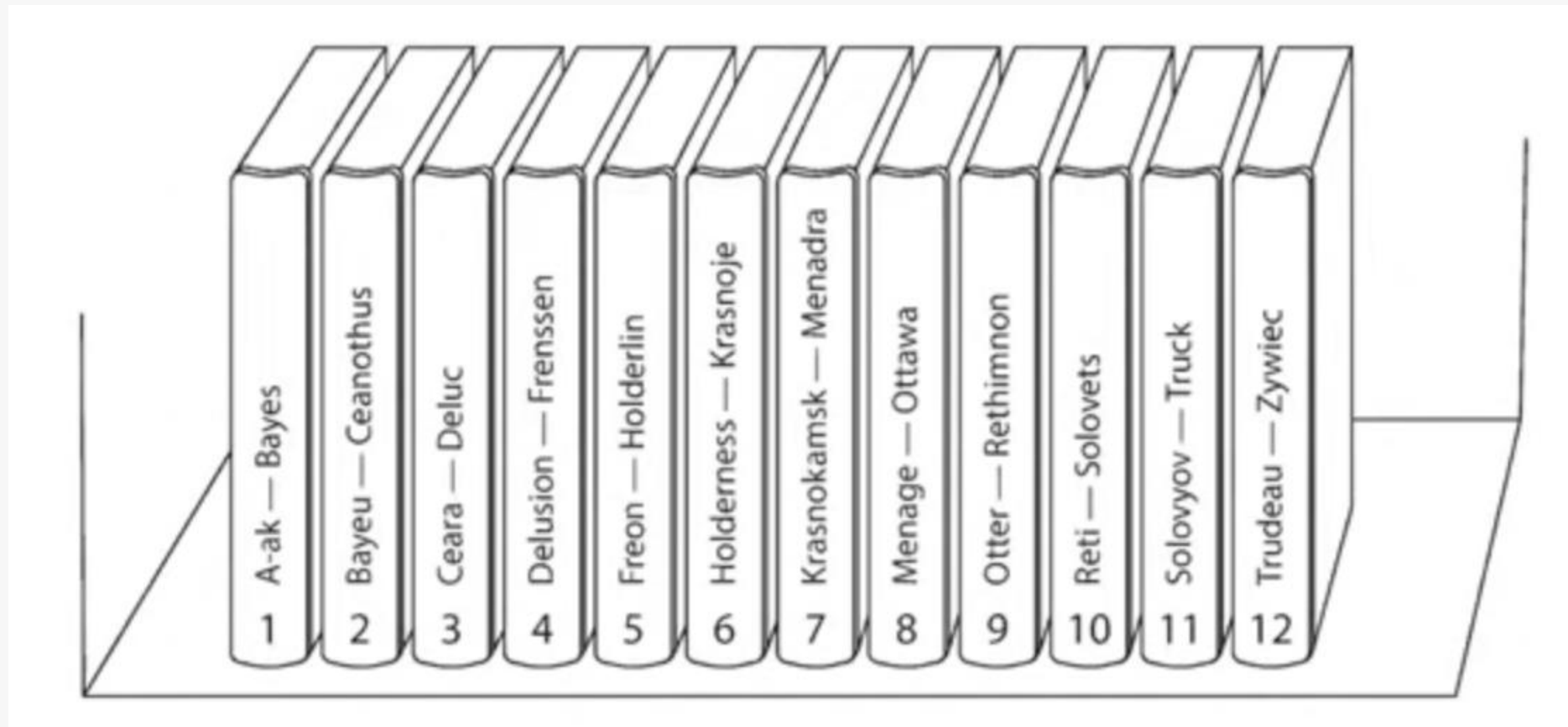
Горизонтальный шардинг.

Стратегии разбиения данных



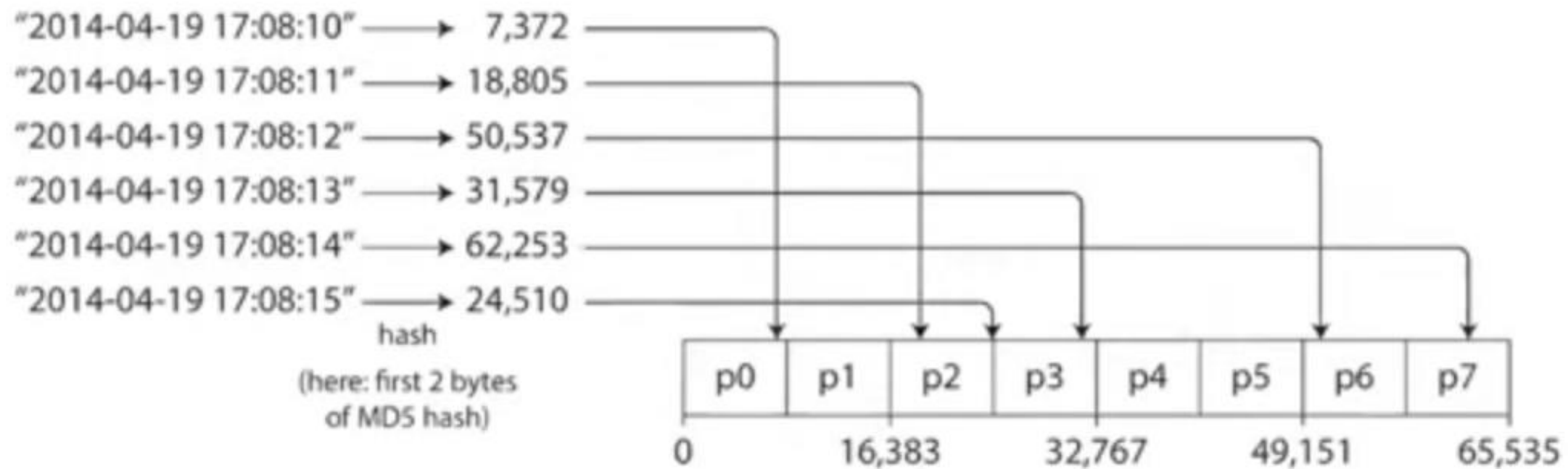
Горизонтальный шардинг. Стратегии разбиения данных

Интервалы ключей



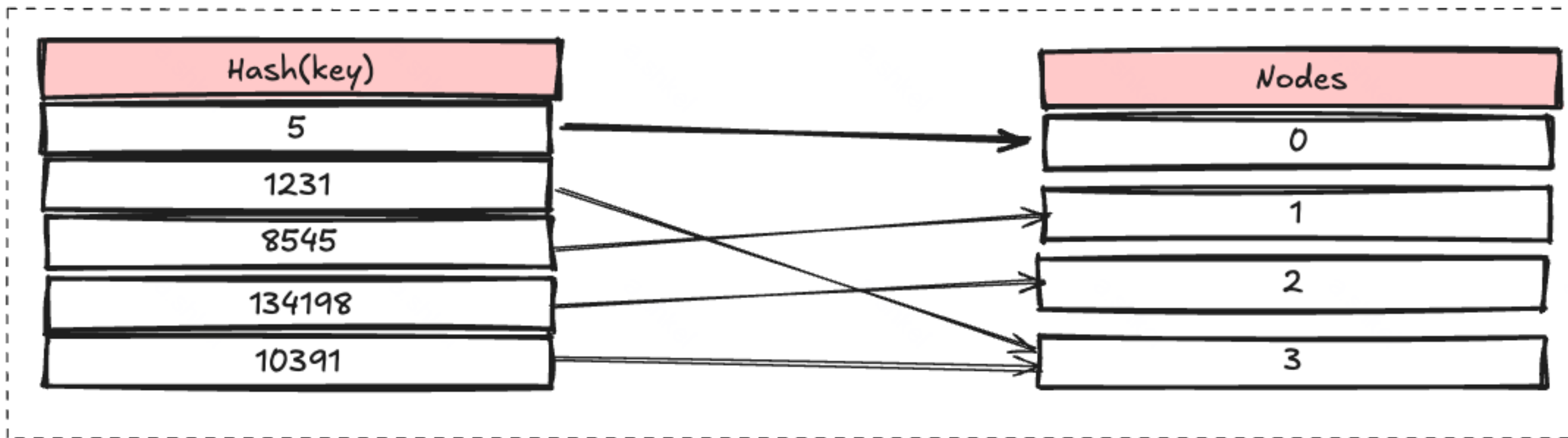
Горизонтальный шардинг. Стратегии разбиения данных

Хэширование всех ключей

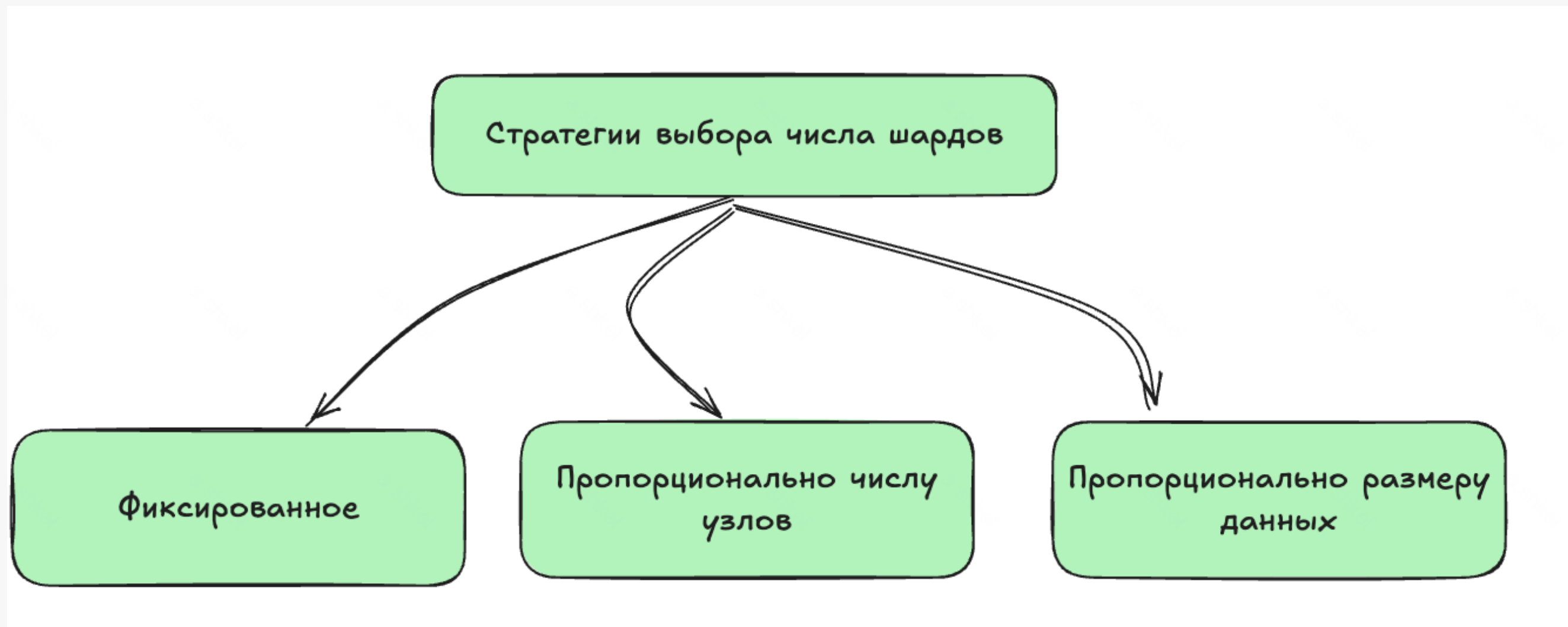


Горизонтальный шардинг. Стратегии разбиения данных

$\text{Hash}(k) \% N$



Стратегии выбора числа шардов



Согласованное хэширование

Согласованное хэширование

ПРОБЛЕМА ПОВТОРНОГО ХЕШИРОВАНИЯ

Если у вас есть n кэширующих серверов, балансирование нагрузки обычно обеспечивается с помощью следующего метода хеширования:

$$\text{serverIndex} = \text{hash}(\text{key}) \% N,$$

где N — размер пула серверов.

Рассмотрим пример того, как это работает. В табл. 5.1 перечислено 4 сервера и 8 строковых ключей вместе с хешами.

Таблица 5.1

Ключ	Хеш	Хеш % 4
ключ 0	18358617	1
ключ 1	26143584	0
ключ 2	18131146	2
ключ 3	35863496	0
ключ 4	34085809	1
ключ 5	27581703	3
ключ 6	38164978	2
ключ 7	22530351	3

Согласованное хэширование

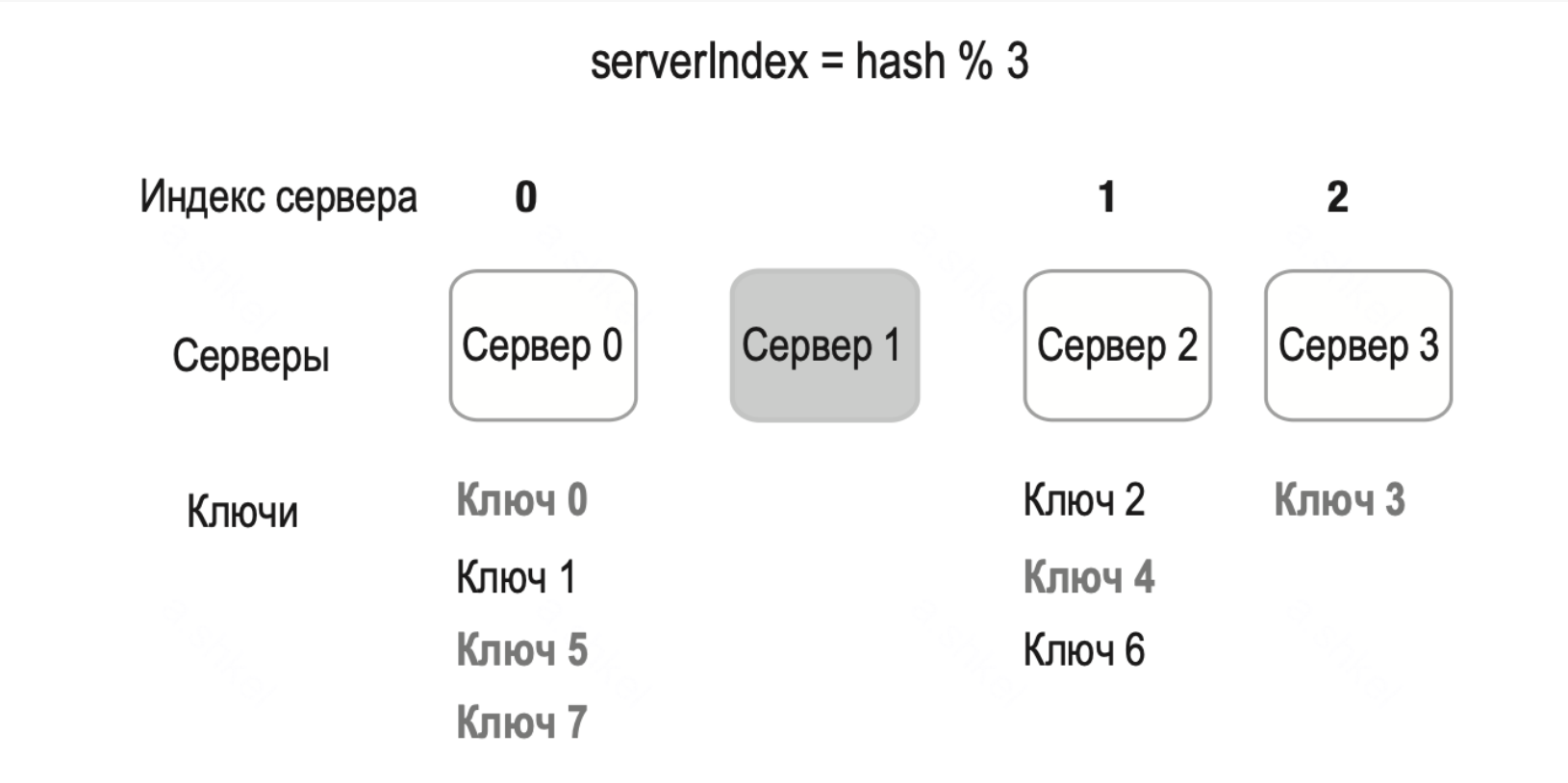
Распределение ключей по 4 серверам

serverIndex = hash % 4				
Индекс сервера	0	1	2	3
Серверы	Сервер 0	Сервер 1	Сервер 2	Сервер 3
Ключи	Ключ 1	Ключ 0	Ключ 2	Ключ 5
	Ключ 3	Ключ 4	Ключ 6	Ключ 7

Согласованное хэширование

1 сервер упал.

Распределение по 3-ем серверам



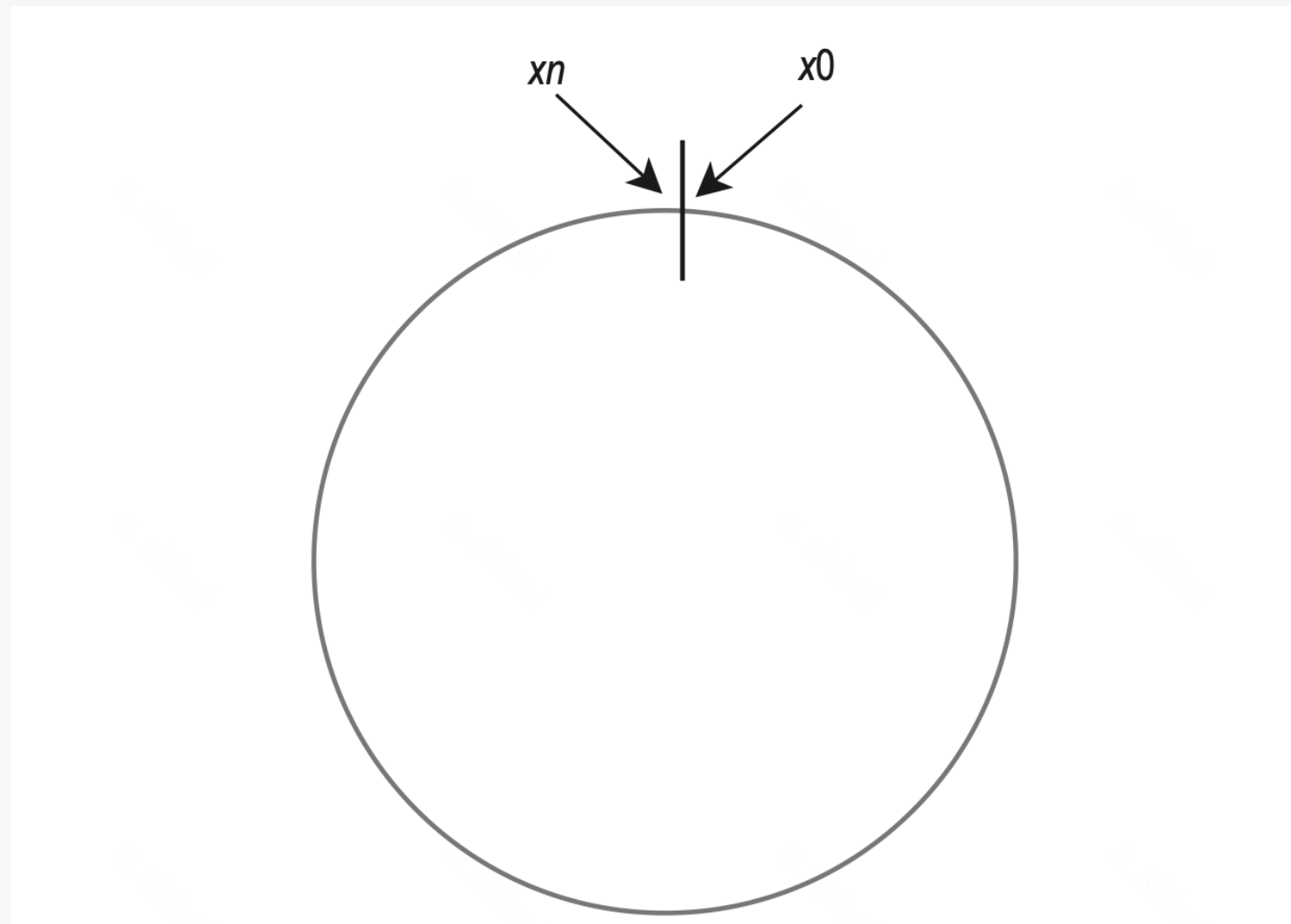
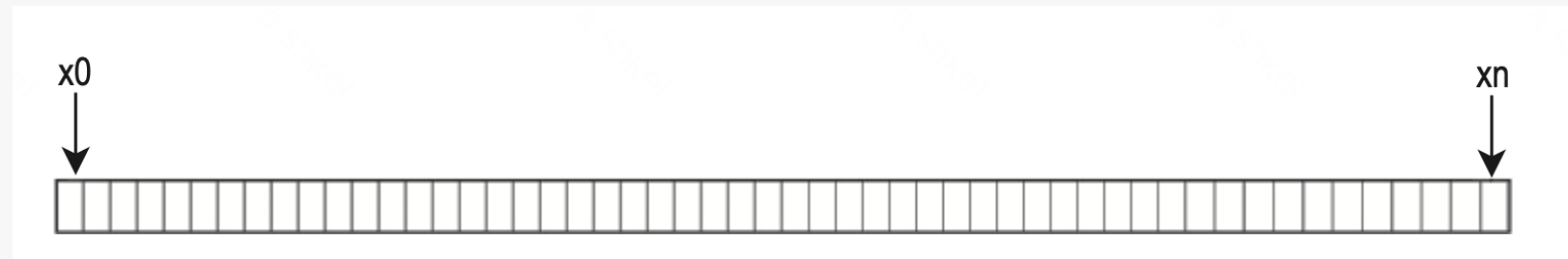
Ключ	Хеш	Хеш % 3
ключ 0	18358617	0
ключ 1	26143584	0
ключ 2	18131146	1
ключ 3	35863496	2
ключ 4	34085809	1
ключ 5	27581703	0
ключ 6	38164978	1
ключ 7	22530351	0

Согласованное хэширование

СОГЛАСОВАННОЕ ХЕШИРОВАНИЕ

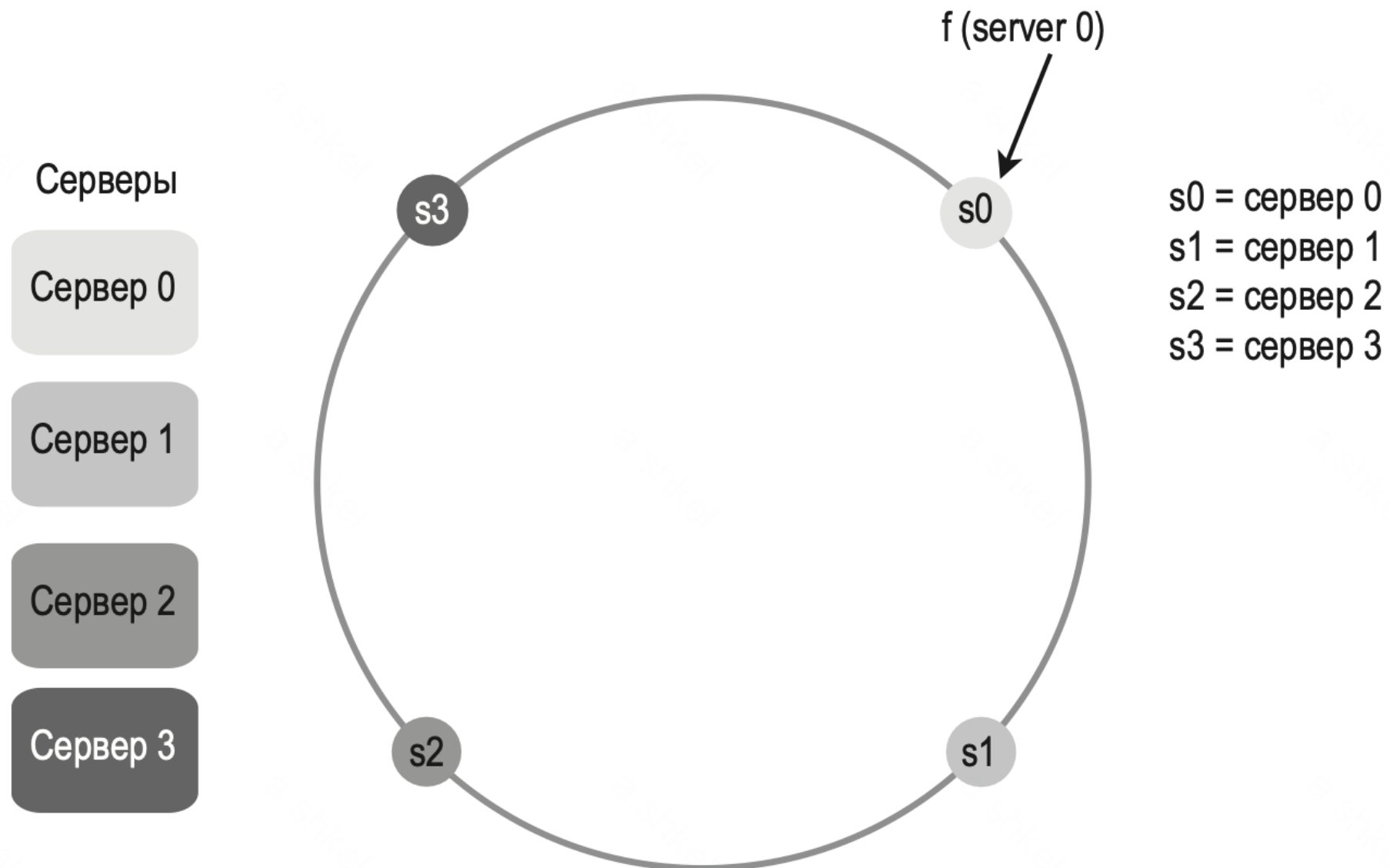
Цитата из Википедии: «Согласованное хеширование (*англ.* consistent hashing) — особый вид хеширования, отличающийся тем, что когда хеш-таблица перестраивается, только K/n ключей в среднем должны быть переназначены, где K — число ключей и n — число слотов. В противоположность этому, в большинстве традиционных хеш-таблиц изменение количества слотов вызывает переназначение почти всех ключей» [1].

Согласованное хэширование. Кольцо хэширования



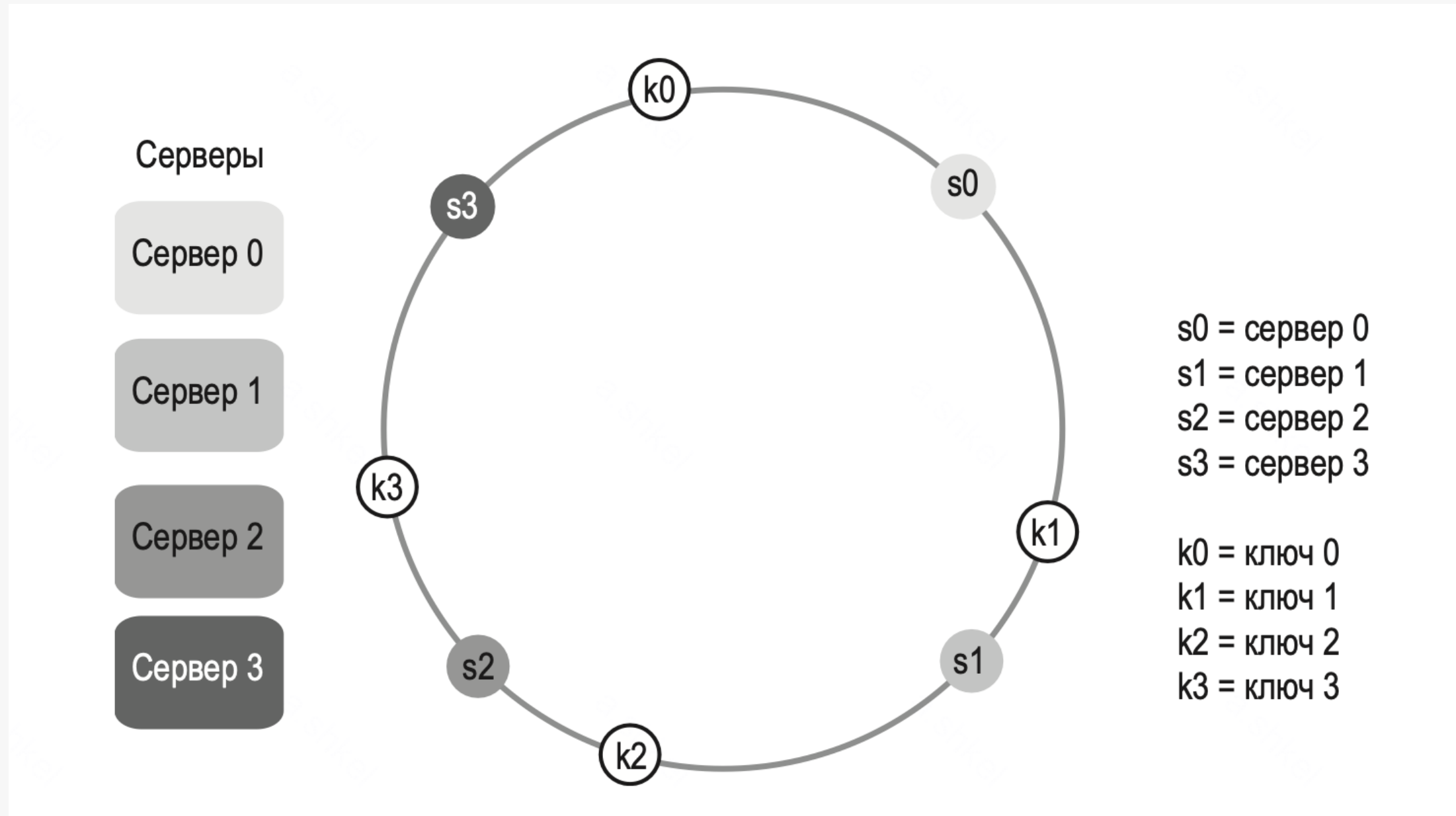
Согласованное хэширование. Кольцо хэширования

Нанесем сервера на кольцо хэширования



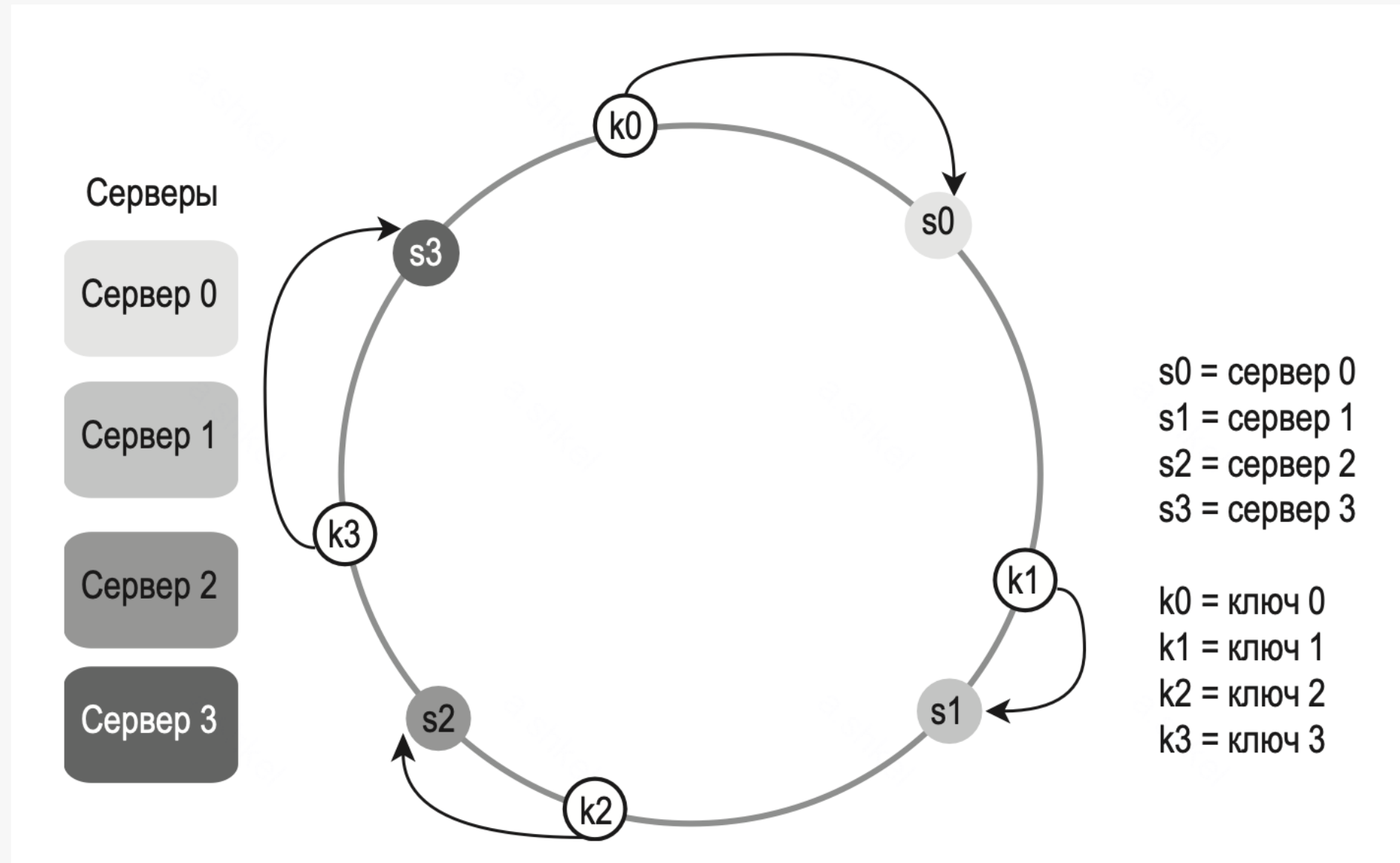
Согласованное хэширование. Кольцо хэширования

Нанесем ключи на кольцо хэширования



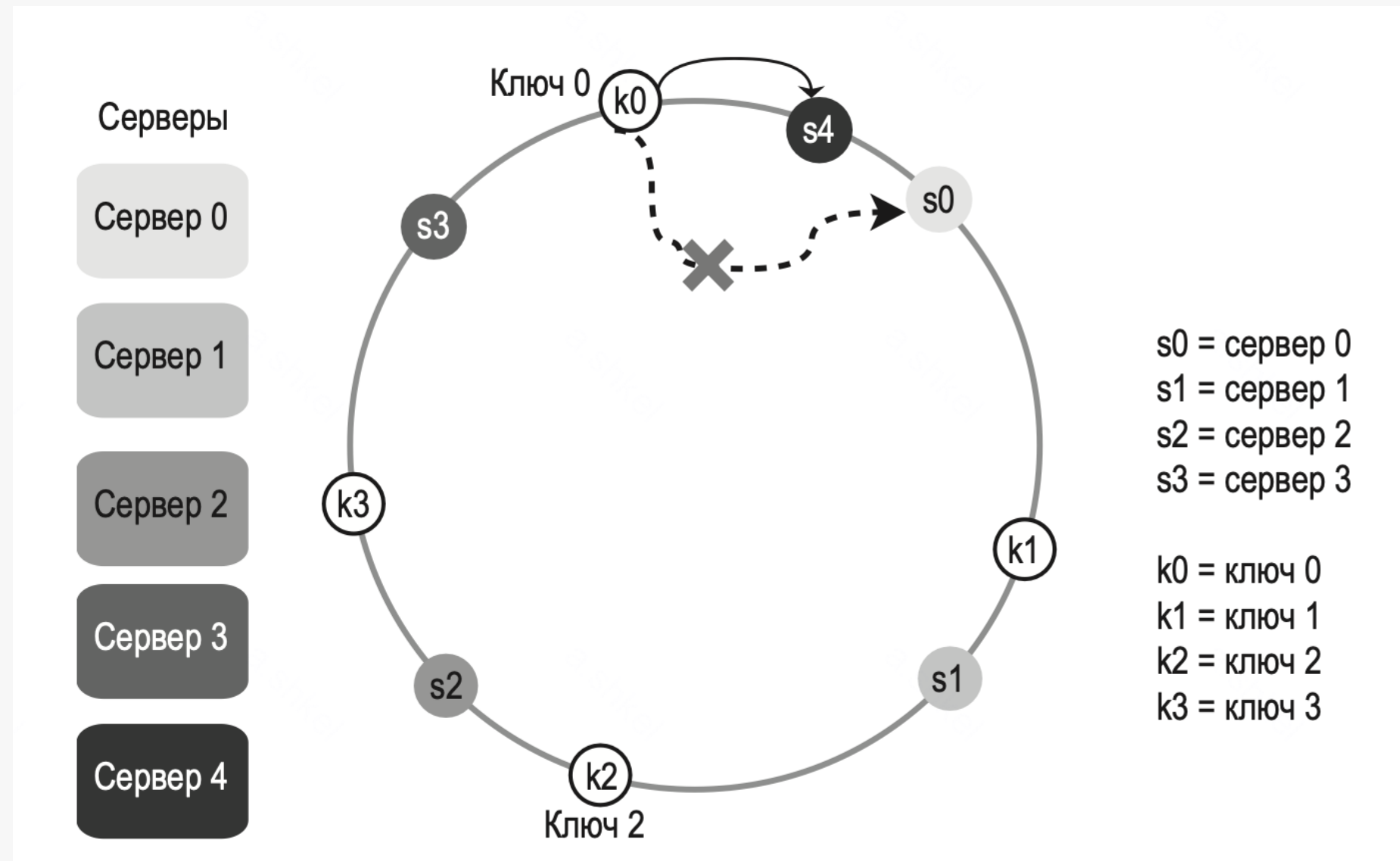
Согласованное хэширование. Кольцо хэширования

Поиск сервера для ключа



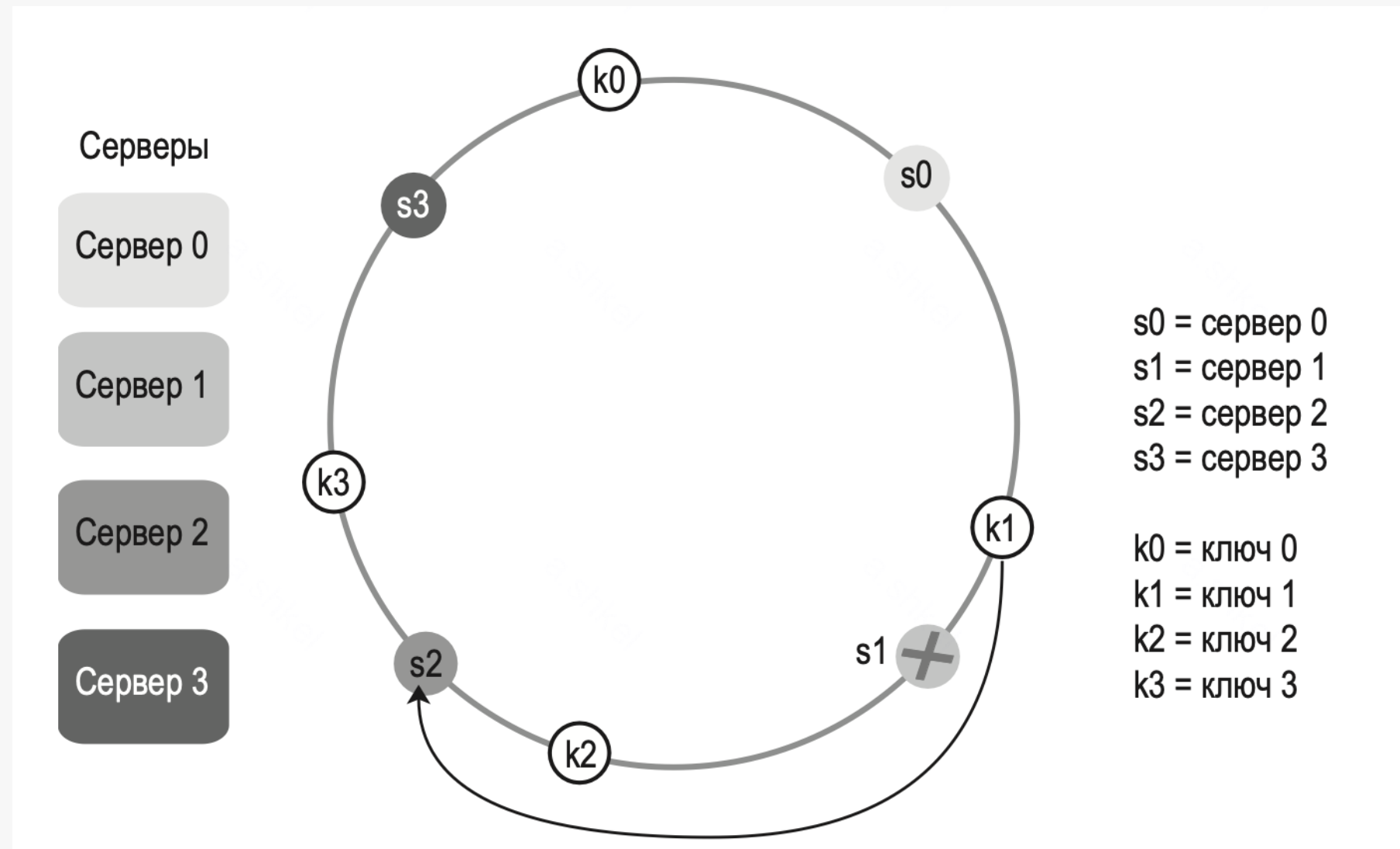
Согласованное хэширование. Кольцо хэширования

Операция добавления сервера



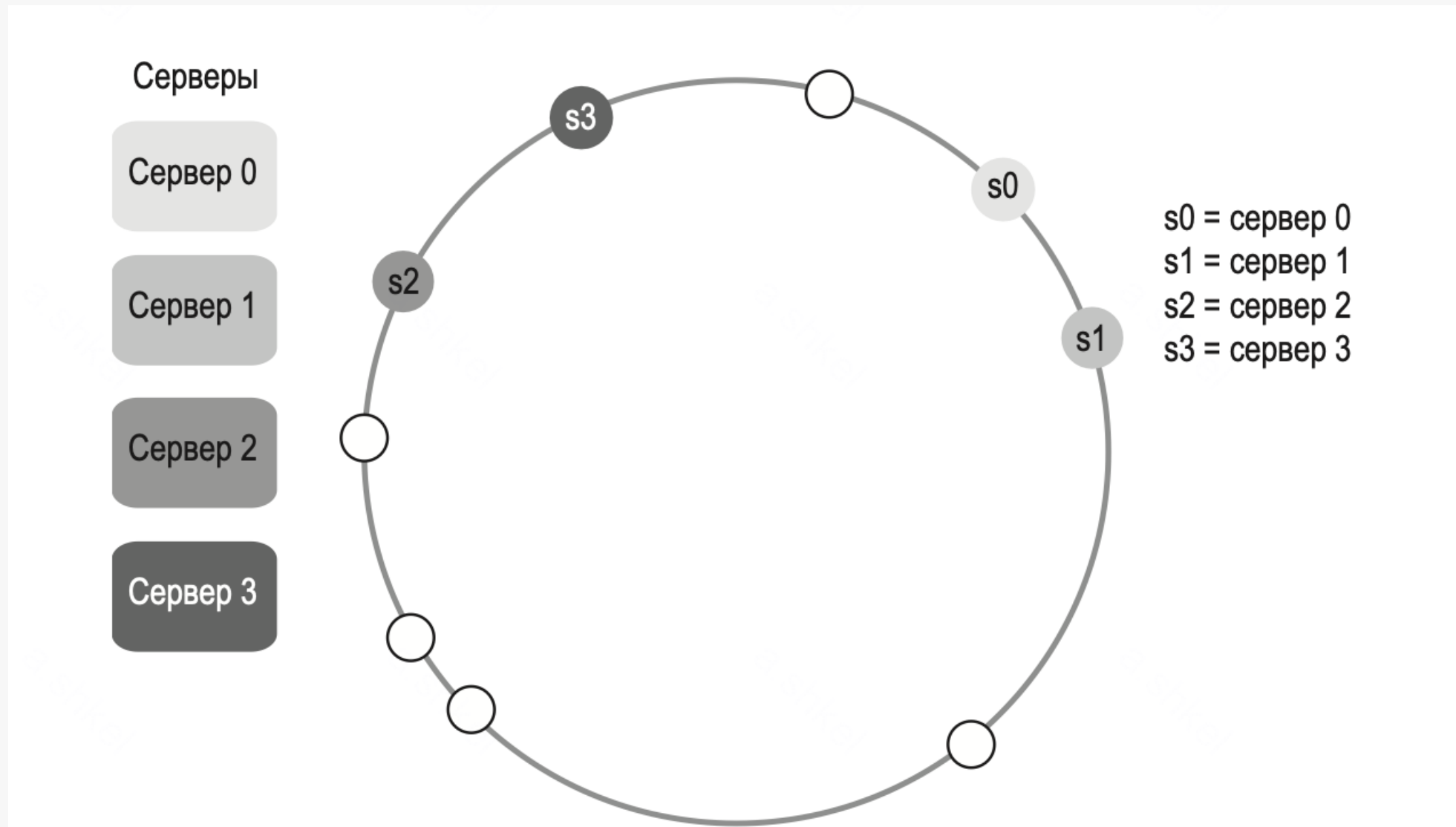
Согласованное хэширование. Кольцо хэширования

Операция удаления сервера



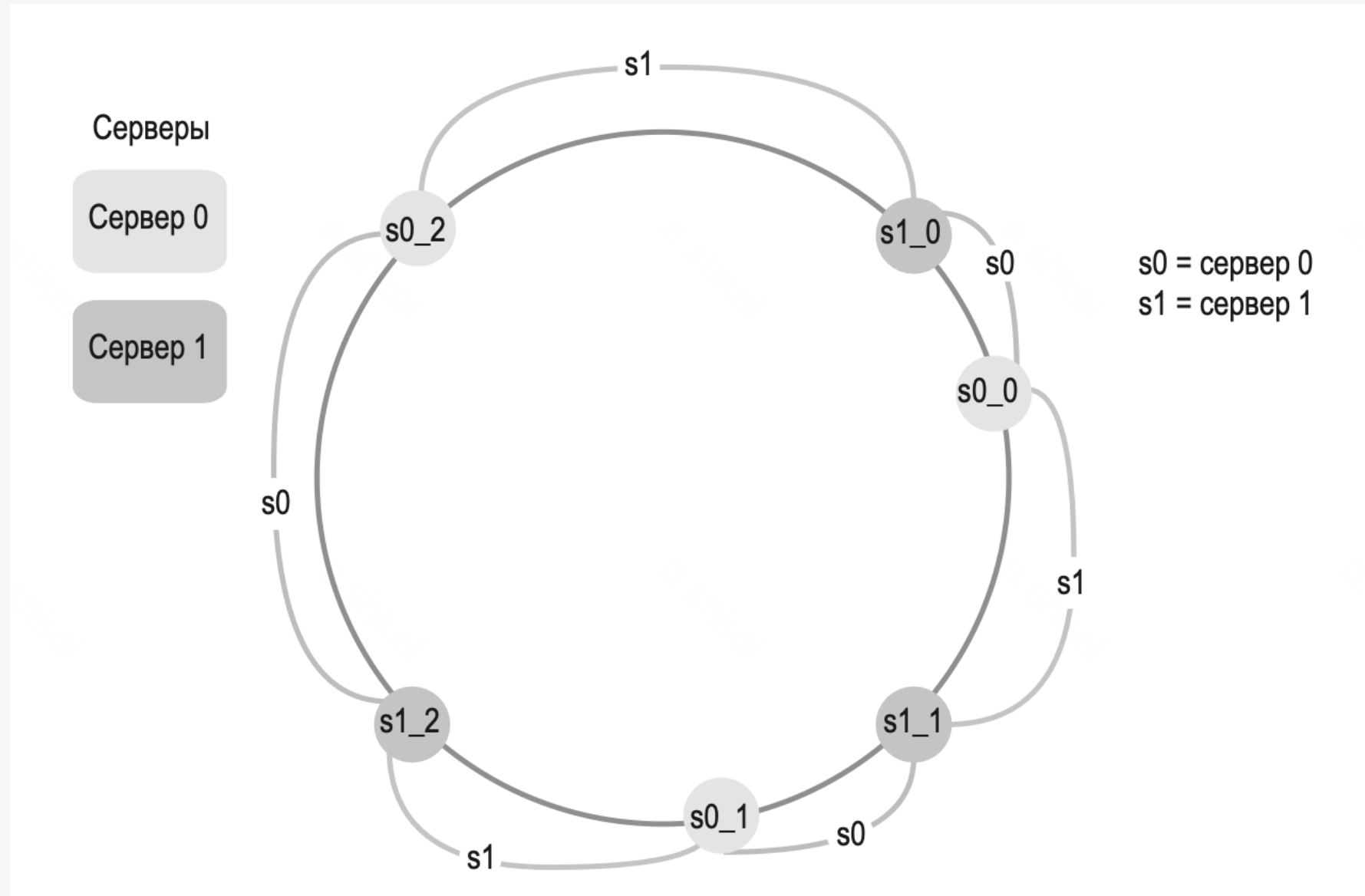
Согласованное хэширование. Кольцо хэширования

Недостатки классического подхода



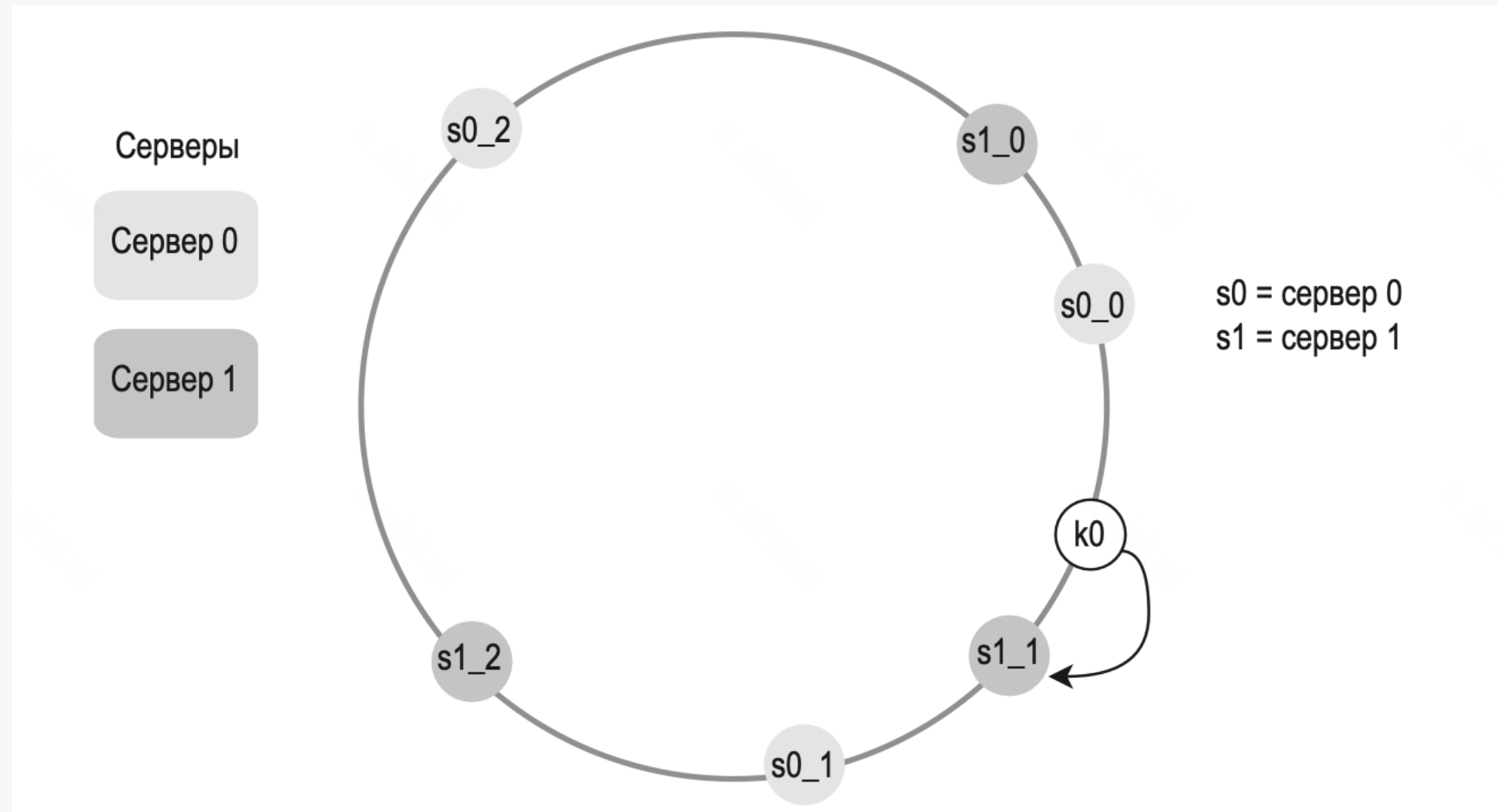
Согласованное хэширование. Кольцо хэширования

Решение в виде виртуальных узлов (vnode)



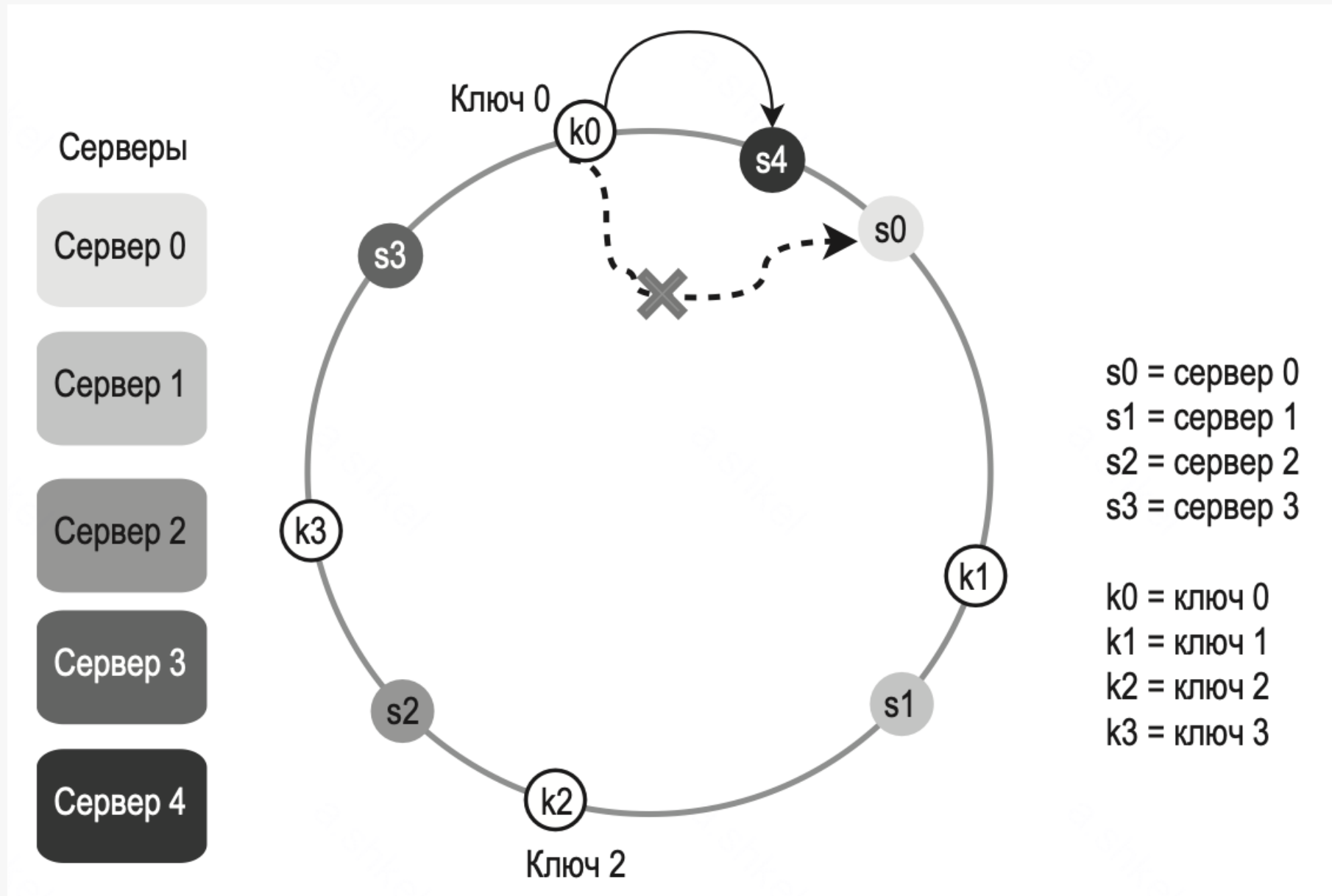
Согласованное хэширование. Кольцо хэширования

Маппинг ключа на ноду при виртуальных узлах



Согласованное хэширование. Кольцо хэширования

Поиск затронутых ключей при добавлении нового сервера



Согласованное хэширование. Кольцо хэширования

Поиск затронутых ключей при падении сервера

